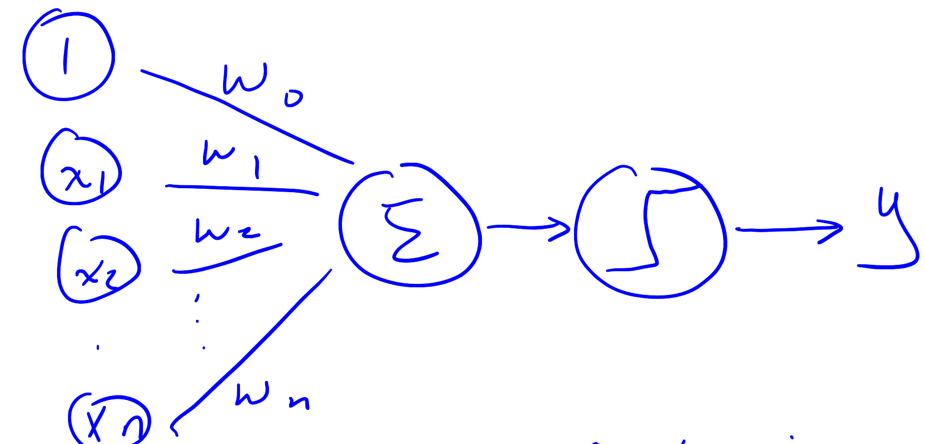
Perceptron



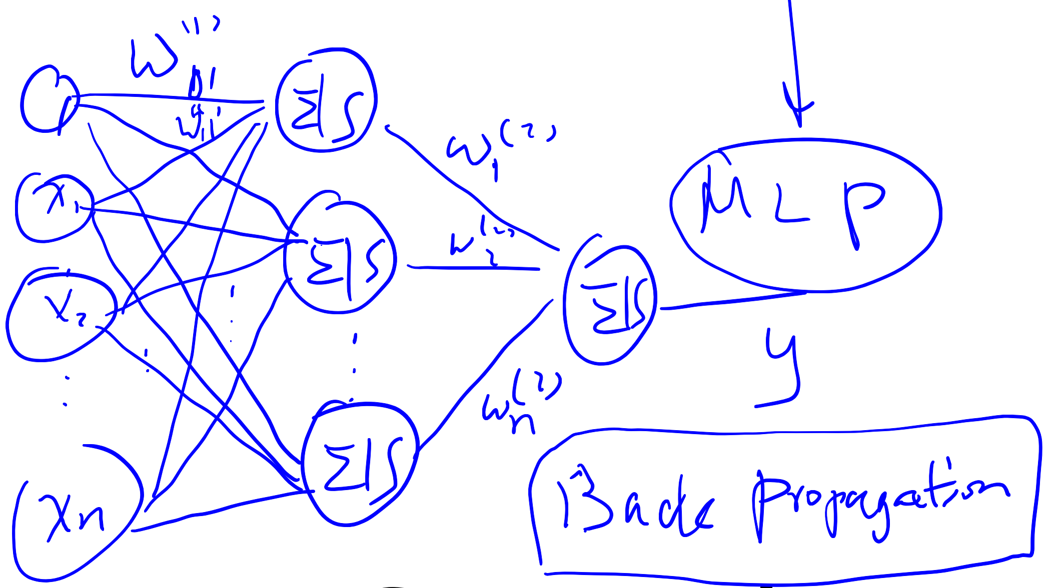
Activation: threshold function

(Sigmoid function)

Linear separable

XOR

Hidden layer



$\boxed{\text{Back Propagation}}$ err $= y - \hat{y}$ $\Rightarrow w^{(n)}_{ij}$

Lecture 6c

Convolutional Neural Networks

Part 1

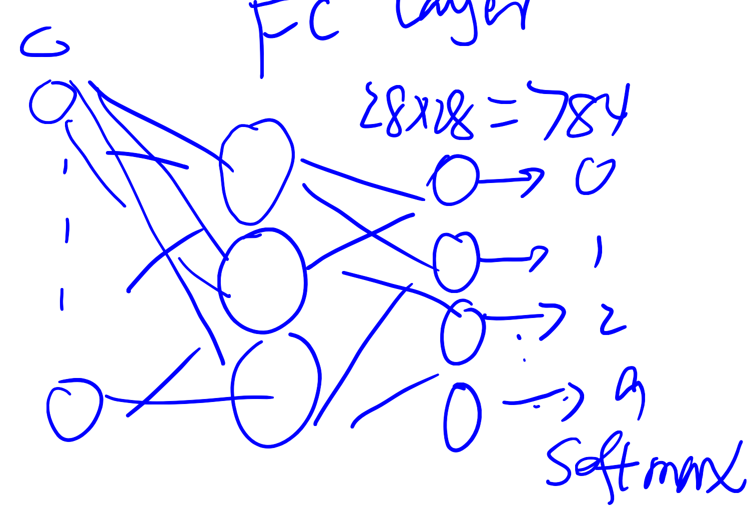
HW#4

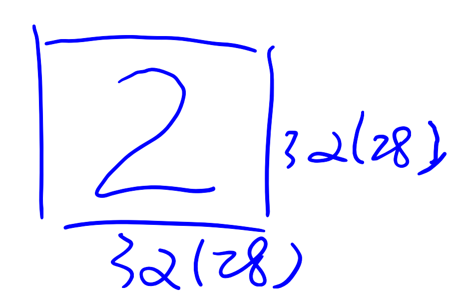
MNIST

Selected slides from Sebastian Raschka

CNN

FC layer





"MLP"

2 classes

CNNs for Image Classification

Kaggle "cat" vs. "dog"

Image Source:
twitter.com%2Fcats&psig=AOvVaw30_o-PCM-K21DiMAJQimQ4&ust=1553887775741551

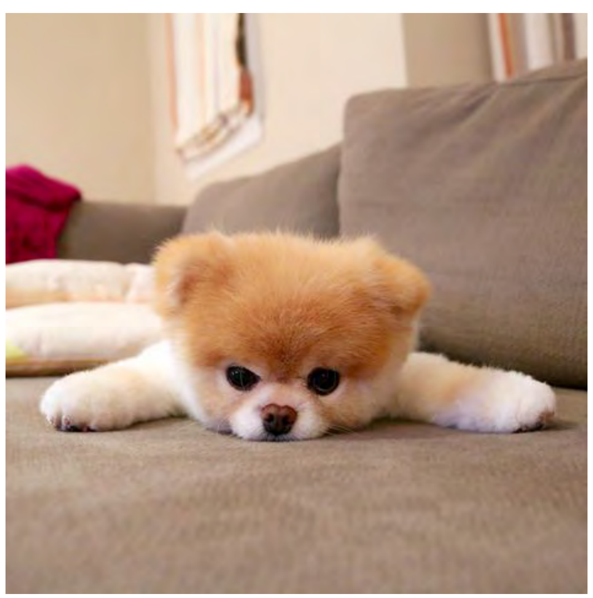
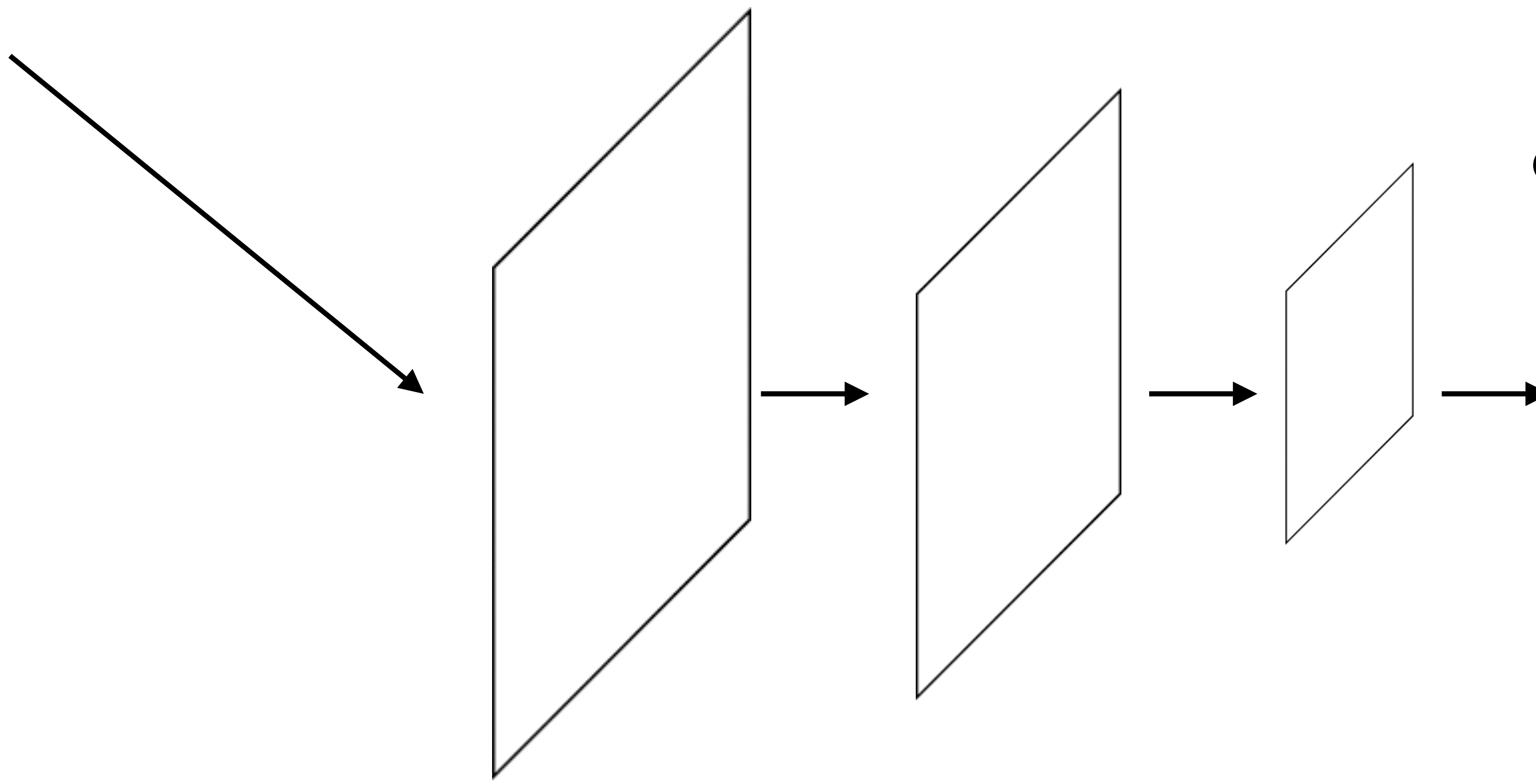


Image Source: https://www.pinterest.com/pin/244742560974520446



output

$p(y{=}cat)$

$1-p$

(dog)

Some Applications of CNNs

Predicting people's age

(160,000 images from the RenRen Social Network)

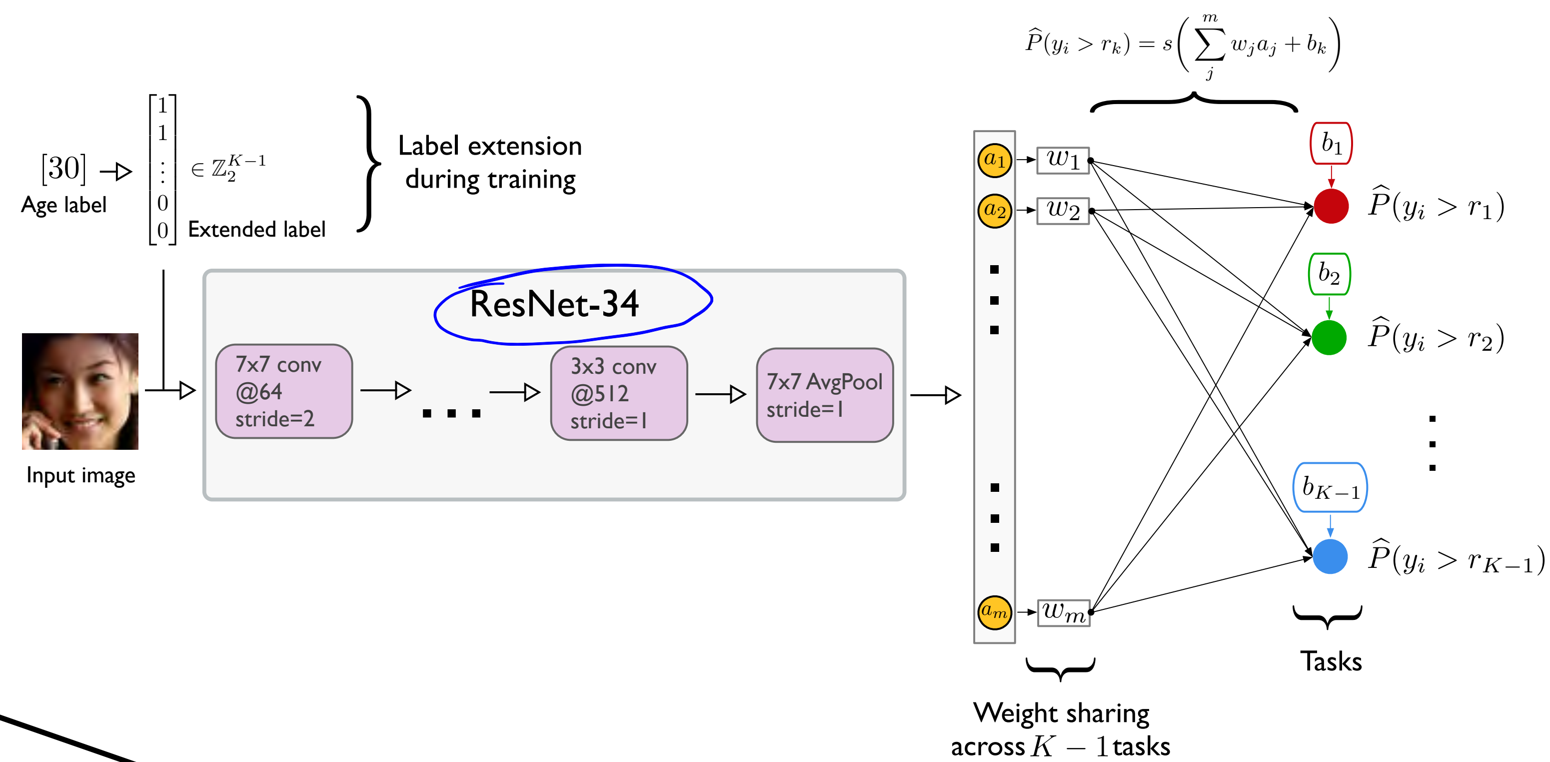


Figure 2. Illustration of the Consistent Rank Logits CNN (CORAL-CNN) used for age prediction. From the estimated probability values, the binary labels are obtained via Eq. (5) and converted to the age label via Eq. (1).

Table 1. Age prediction errors on the test sets *without* task importance weighting.

Method	Random Seed	MORPH-2		AFAD		UTKFace		CACD	
		MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
CE-CNN	0	3.40	4.88	3.98	5.55	6.57	9.16	6.18	8.86
	1	3.39	4.87	4.00	5.57	6.24	8.69	6.10	8.79
	2	3.37	4.87	3.96	5.50	6.29	8.78	6.13	8.87
	AVG ± SD	3.39 ± 0.02	4.89 ± 0.01	3.98 ± 0.02	5.54 ± 0.04	6.37 ± 0.18	8.88 ± 0.25	6.14 ± 0.04	8.84 ± 0.04
OR-CNN (Niu et al., 2016)	0	2.98	4.26	3.66	5.10	5.71	8.11	5.53	7.91
	1	2.98	4.26	3.69	5.13	5.80	8.12	5.53	7.98
	2	2.96	4.20	3.68	5.14	5.71	8.11	5.49	7.89
	AVG ± SD	2.97 ± 0.01	4.24 ± 0.03	3.68 ± 0.02	5.13 ± 0.02	5.74 ± 0.05	8.08 ± 0.06	5.52 ± 0.02	7.93 ± 0.05
CORAL-CNN (ours)	0	2.68	3.75	3.49	4.82	5.46	7.61	5.56	7.80
	1	2.63	3.66	3.46	4.83	5.46	7.63	5.37	7.64
	2	2.61	3.64	3.52	4.91	5.48	7.63	5.25	7.53
	AVG ± SD	**2.64 ± 0.04**	**3.68 ± 0.06**	**3.49 ± 0.03**	**4.85 ± 0.05**	**5.47 ± 0.01**	**7.62 ± 0.01**	**5.39 ± 0.16**	**7.66 ± 0.14**

Cao, Wenzhi, Vahid Mirjalili, and Sebastian Raschka. "Consistent Rank Logits for Ordinal Regression with Convolutional Neural Networks." *arXiv preprint arXiv:1901.07884* (2019).

Some Applications of CNNs

Dermatologist-level classification of skin cancer

An artificial intelligence trained to classify images of skin lesions as benign lesions or malignant skin cancers achieves the accuracy of board-certified dermatologists.

In this work, we pretrain a deep neural network at general object recognition, then fine-tune it on a dataset of ~130,000 skin lesion images comprised of over 2000 diseases.

FULL NATURE ARTICLE >

OPEN-ACCESS PDF >

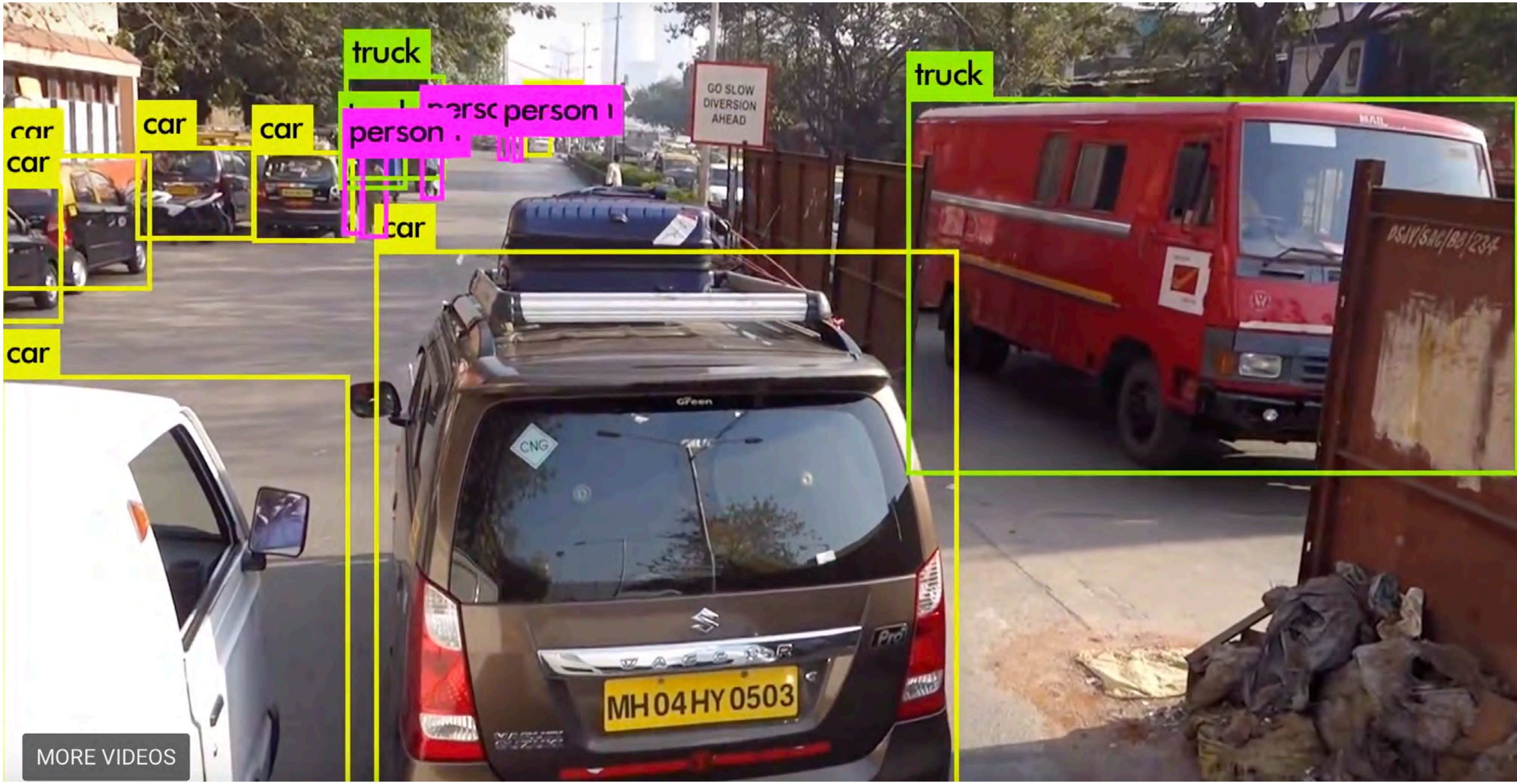
Esteva, Andre, Brett Kuprel, Roberto A. Novoa, Justin Ko, Susan M. Swetter, Helen M. Blau, and Sebastian Thrun. "Dermatologist-level classification of skin cancer with deep neural networks." *Nature* 542, no. 7639 (2017): 115.

HW #5

Object Detection

RCNN

image → [candidates] → [classifier] →



YOLO (CNN)

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 779-788).

→ CNN →

Semantic → classes (pixel)

Object Segmentation

instance Segmentation

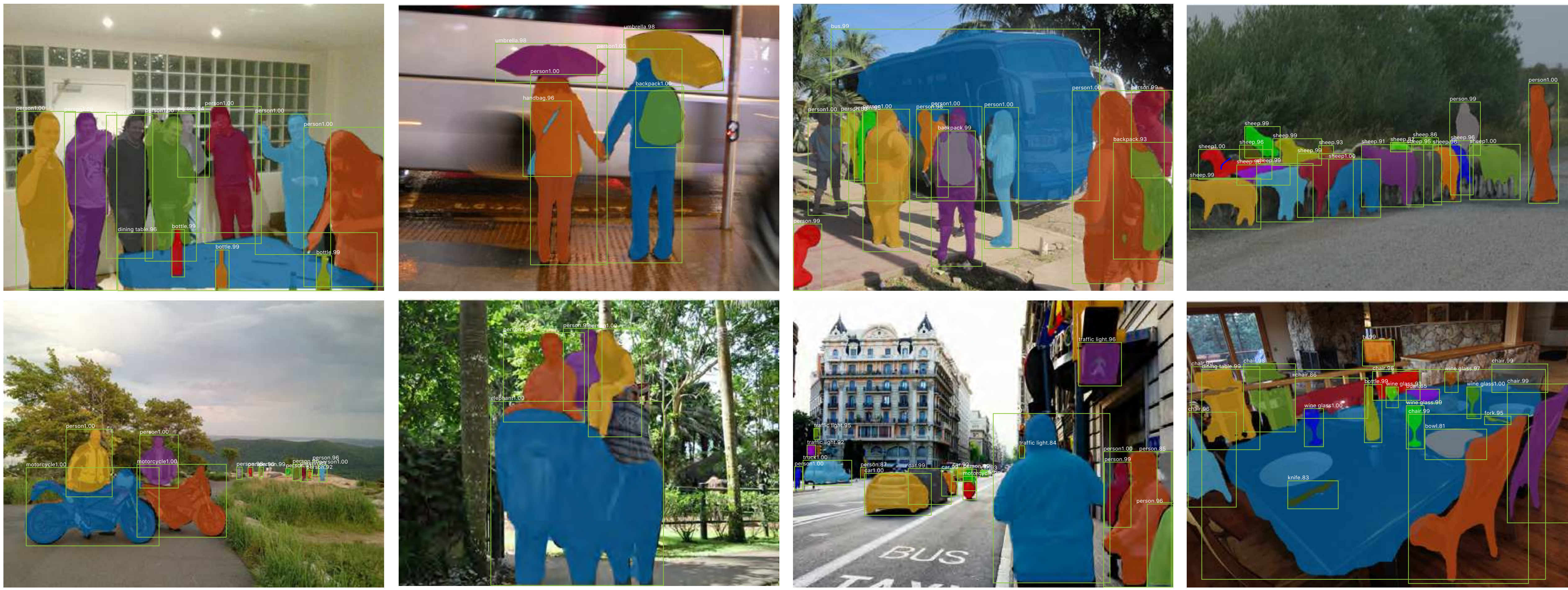


Figure 2. **Mask R-CNN** results on the COCO test set. These results are based on ResNet-101 [15], achieving a *mask* AP of 35.7 and running at 5 fps. Masks are shown in color, and bounding box, category, and confidences are also shown.

He, Kaiming, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. "Mask R-CNN." In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2961-2969. 2017.

Why Image Classification is Hard

ImageNet (1000 classes)

Different lighting, contrast, viewpoints, etc.

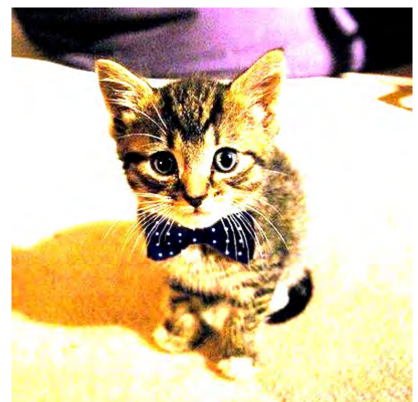
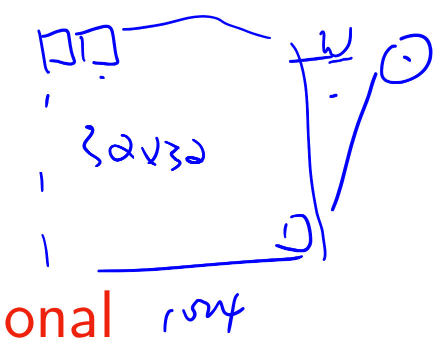


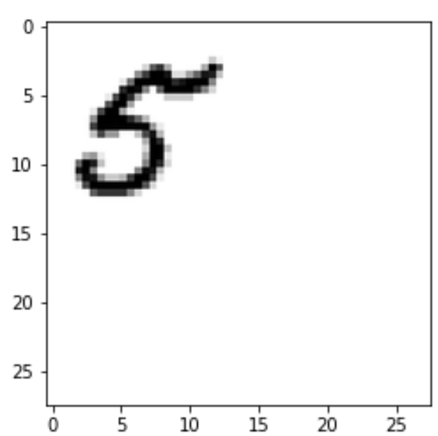
Image Source: twitter.com%2Fcats&psig=AOvVaw30_o-PCM-K21DiMAJQimQ4&ust=1553887775741551

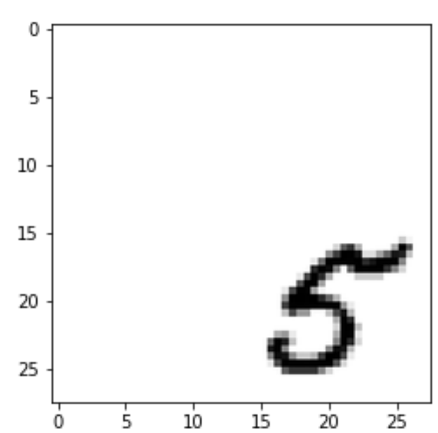
distortion

Image Source: https://www.123rf.com/photo_76714328_side-view-of-tabby-cat-face-over-white.html



Or even simple translation





This is hard for traditional methods like multi-layer perceptrons, because the prediction is basically based on a sum of pixel intensities

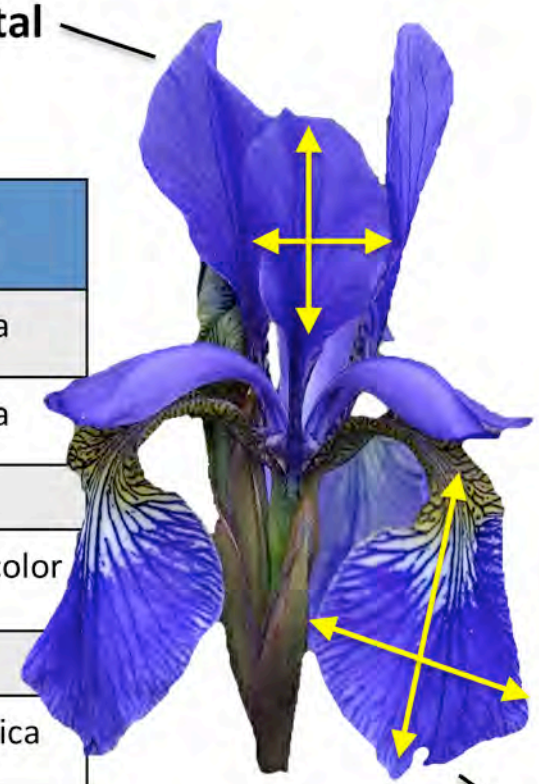
Traditional Approaches

handcraft

a) Use hand-engineered features

Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
			...		
50	6.4	3.5	4.5	1.2	Versicolor
			...		
150	5.9	3.0	5.0	1.8	Virginica



Features
(attributes, measurements, dimensions)

Class labels
(targets)

Udacity

Car / No car

raw pixel

↓

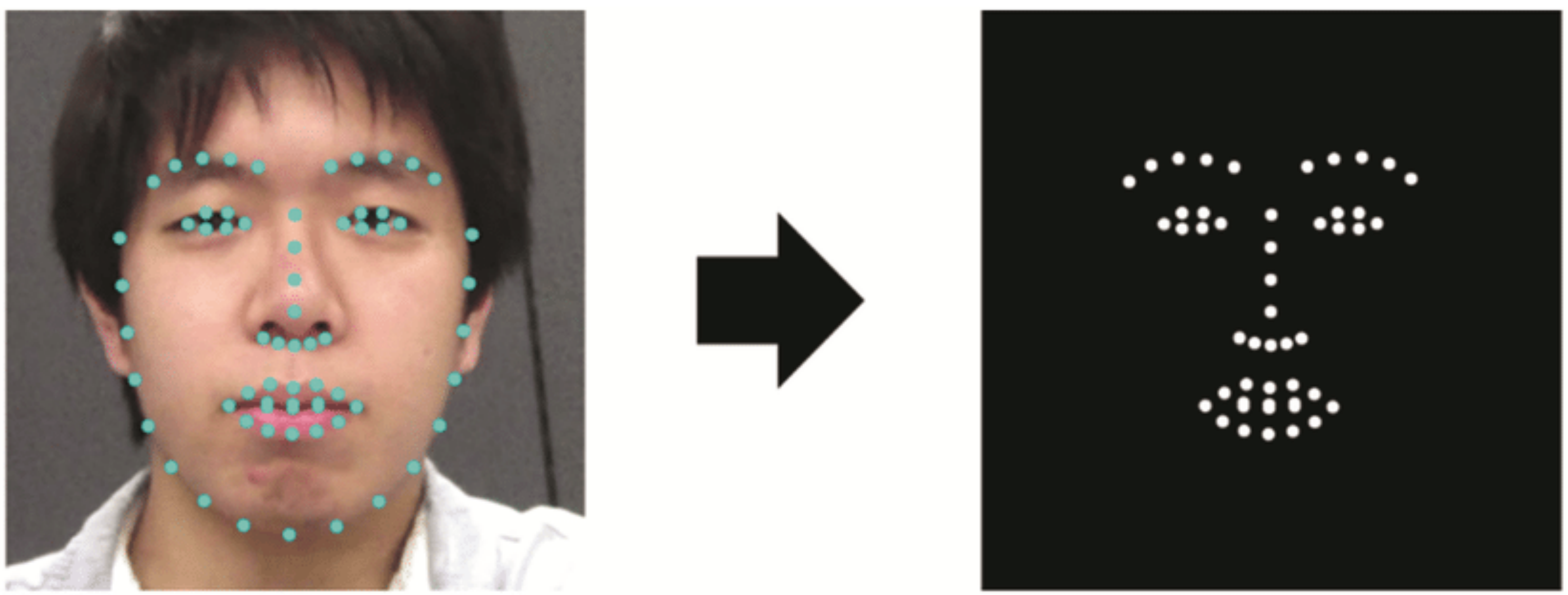
Feature extraction
(HOG)
outline shape

↓

SVM
(classifier)

Traditional Approaches

a) Use hand-engineered features



(a) Detected facial keypoints (b) Facial organ keypoints

Sasaki, K., Hashimoto, M., & Nagata, N. (2016). Person Invariant Classification of Subtle Facial Expressions Using Coded Movement Direction of Keypoints. In *Video Analytics. Face and Facial Expression Recognition and Audience Measurement* (pp. 61-72). Springer, Cham.

Traditional Approaches

b) Preprocess images (centering, cropping, etc.)

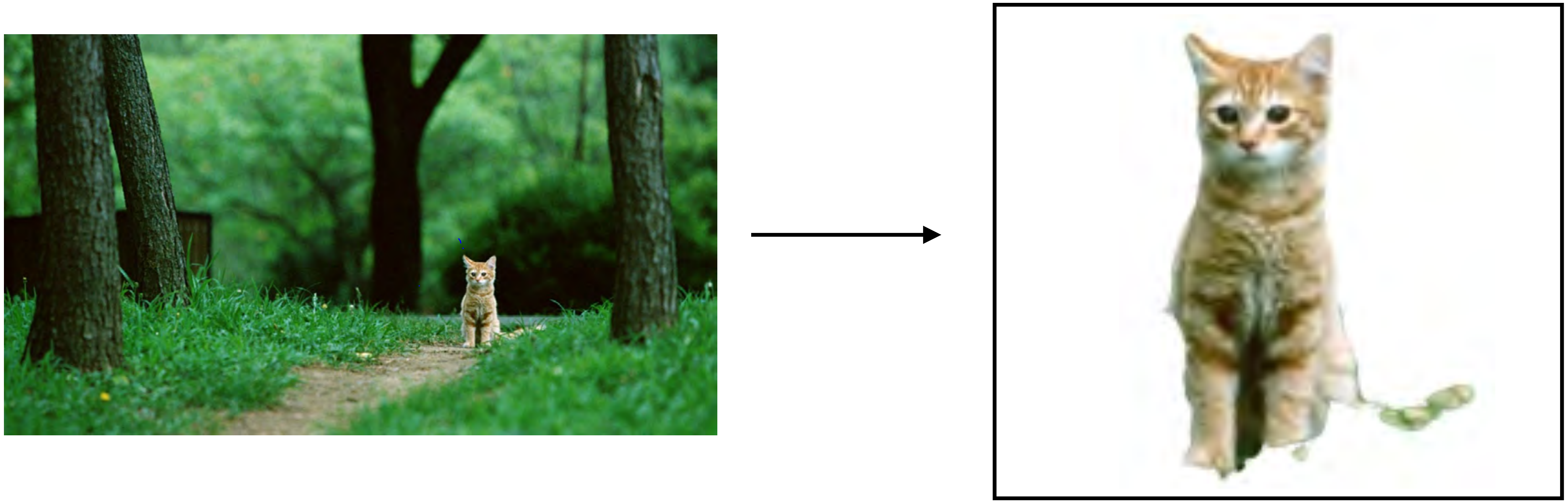


Image Source: https://www.tokkoro.com/2827328-cat-animals-nature-feline-park-green-trees-grass.html

Main Concepts Behind (image)

Convolutional Neural Networks (MLP)

- **Sparse-connectivity**: A single element in the feature map is connected to only a small patch of pixels. (This is very different from connecting to the whole input image, in the case of multi-layer perceptrons.)

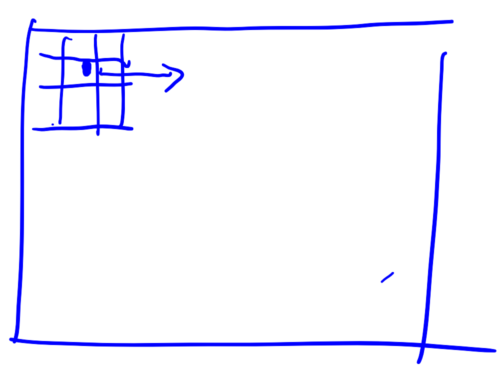
filter

- **Parameter-sharing**: The same weights are used for different patches of the input image.

convolution layer

mask

image → filter



3x3

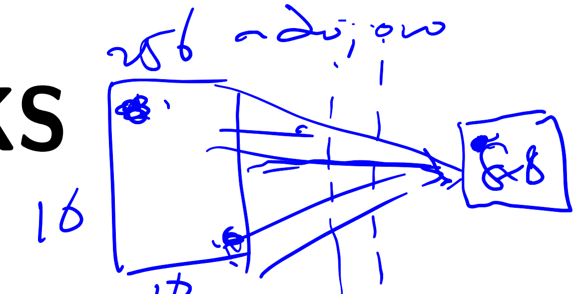
5x5

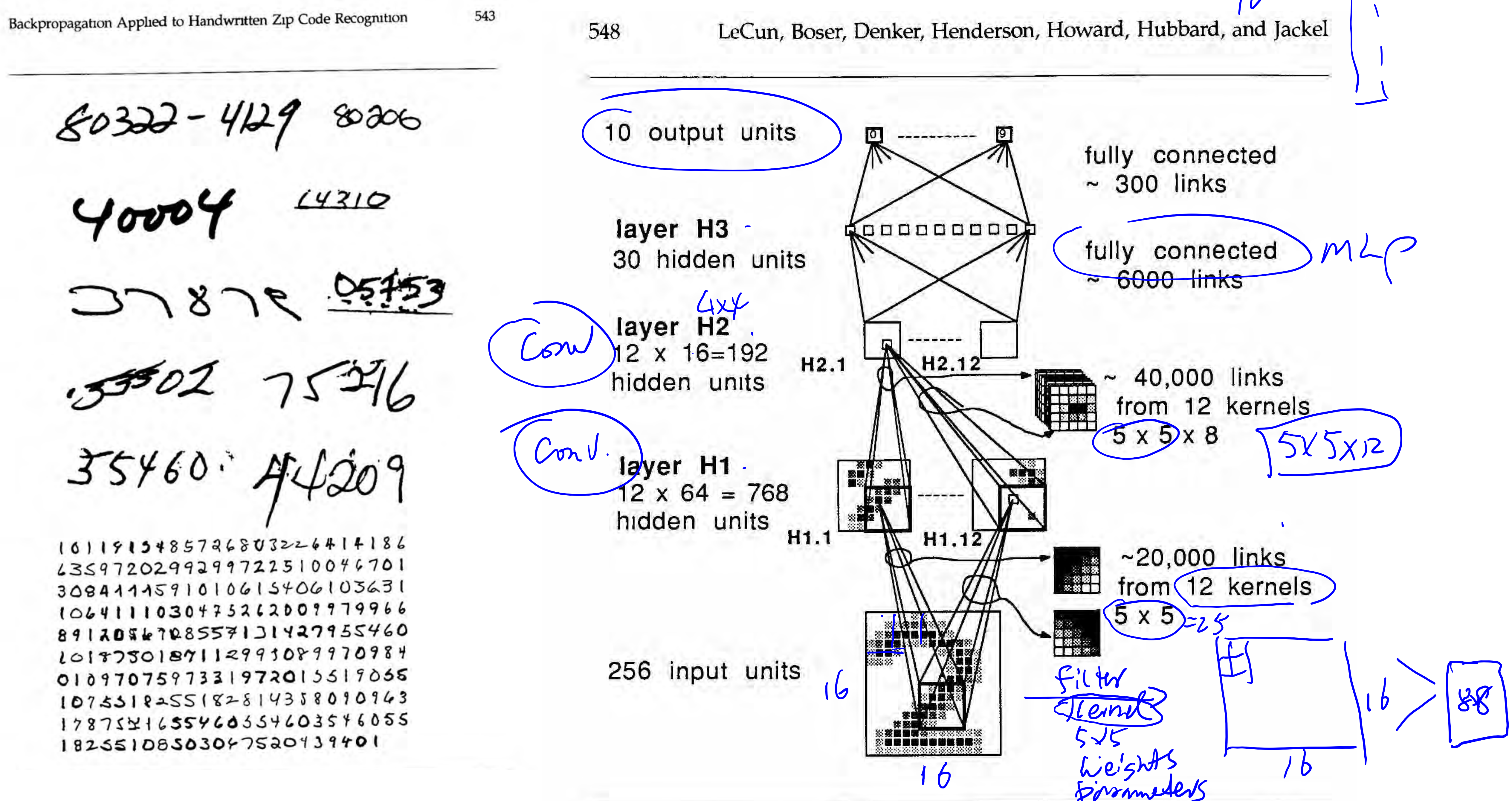
slide through, update pixel value

Sobel filter
(edge detection)

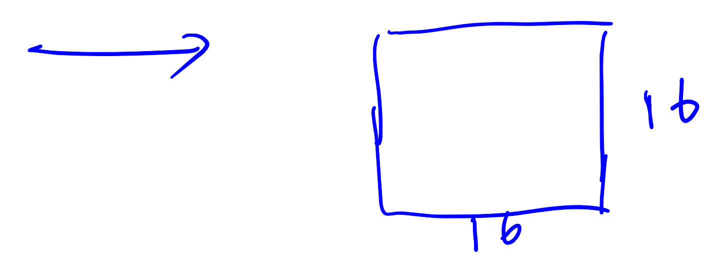
Image Source: https://www.tokkoro.com/2827328-cat-animals-nature-feline-park-green-trees-grass.html

Convolutional Neural Networks





Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel: Backpropagation Applied to Handwritten Zip Code Recognition, Neural Computation, 1(4):541-551, Winter 1989.



Convolutional Neural Networks

wide

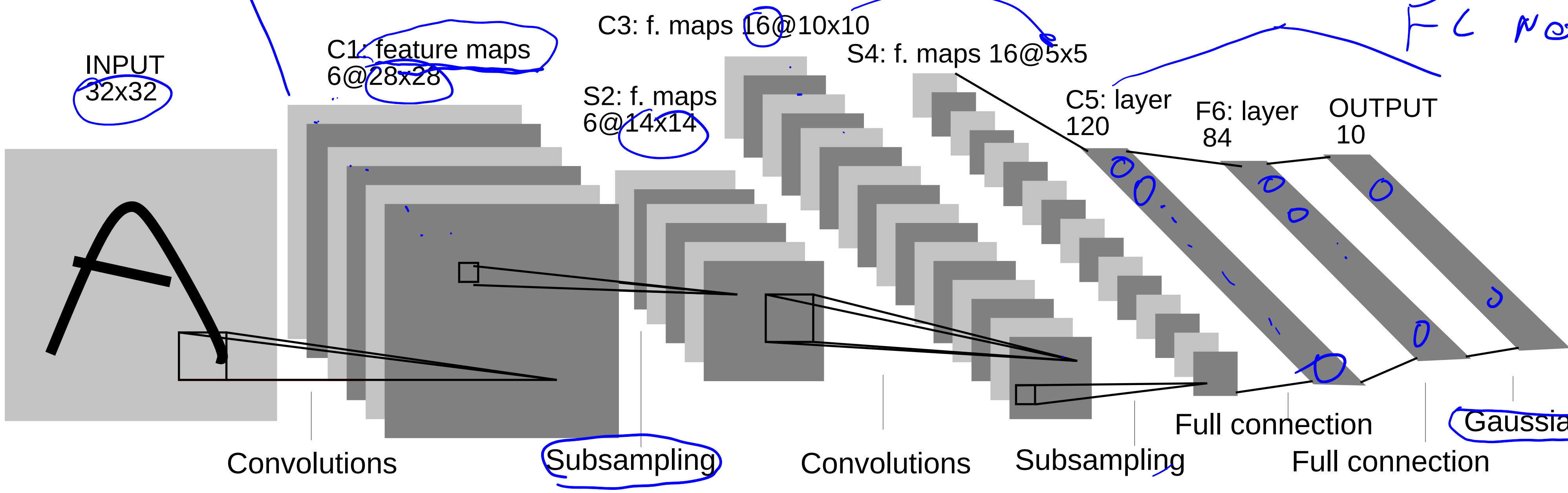
deep

PROC. OF THE IEEE, NOVEMBER 1998 7

nodes

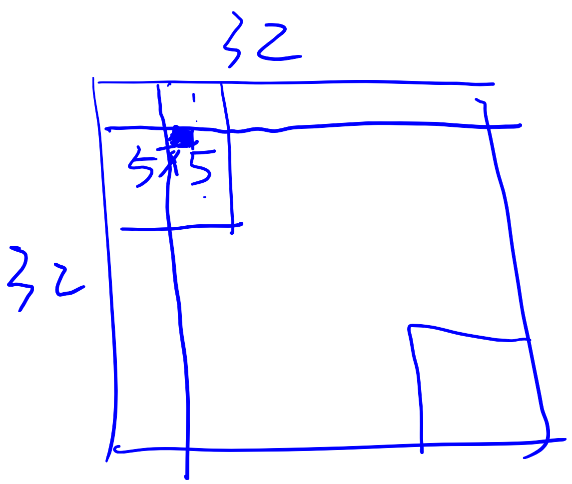
16 x 25 = 400

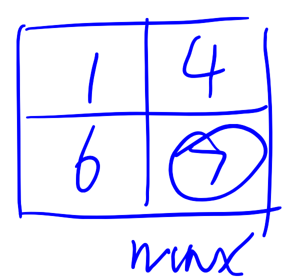
FC network + output

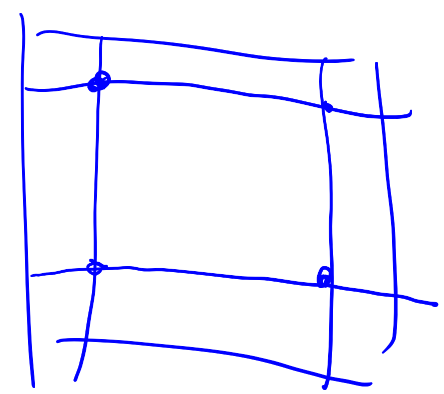


pooling

Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.







"Hyperparameters"

32, 64

Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner: Gradient Based Learning Applied to Document Recognition, Proceedings of IEEE, 86(11):2278–2324, 1998.

Convolutional Neural Networks

feature extractor

PROC. OF THE IEEE, NOVEMBER 1998 7

Number of feature detectors

Size of the resulting layers

classifier

Multi-layer perceptron

FC Network

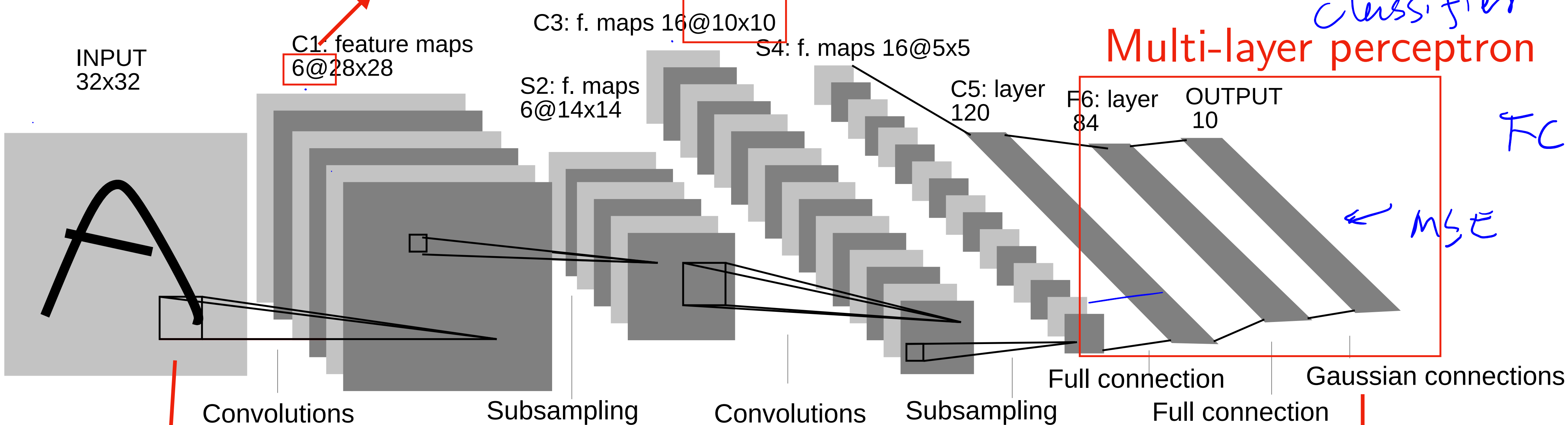


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

nowadays called "pooling"

basically a fully-connected layer + MSE loss (nowadays better to use fc-layer + softmax + cross entropy

"Feature detectors" (weight matrices) that are being reused ("weight sharing") => also called "kernel" or "filter"

Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner: Gradient Based Learning Applied to Document Recognition, Proceedings of IEEE, 86(11):2278–2324, 1998.

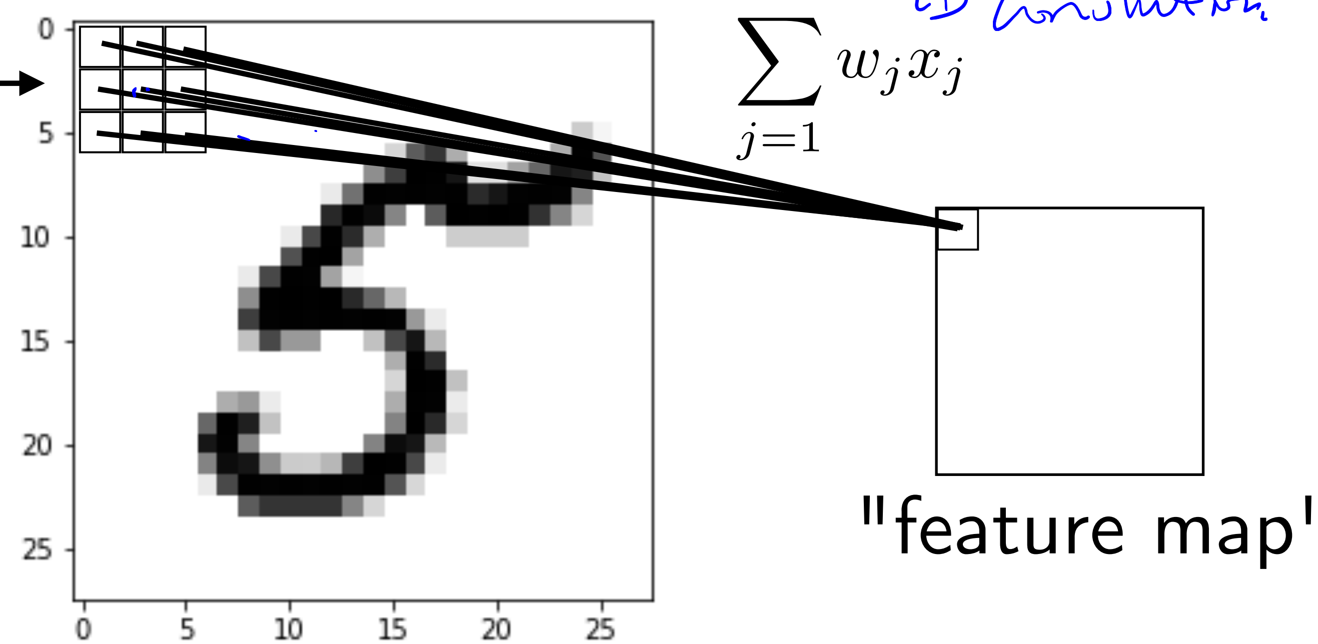
Weight Sharing

A "feature detector" (kernel) slides over the inputs to generate a feature map

The pixels are referred to as "receptive field"

$$\sum_{j=1}^{9} w_j x_j$$

2D Convolution

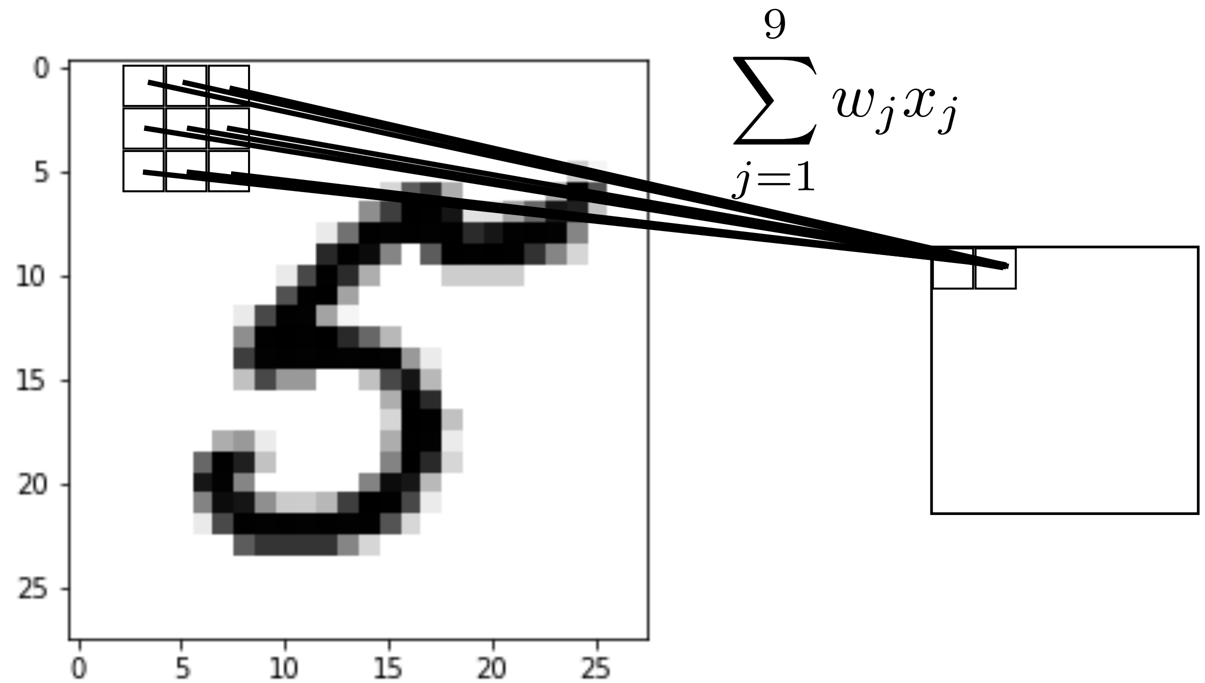


Rationale: A feature detector that works well in one region may also work well in another region

Plus, it is a nice reduction in parameters to fit

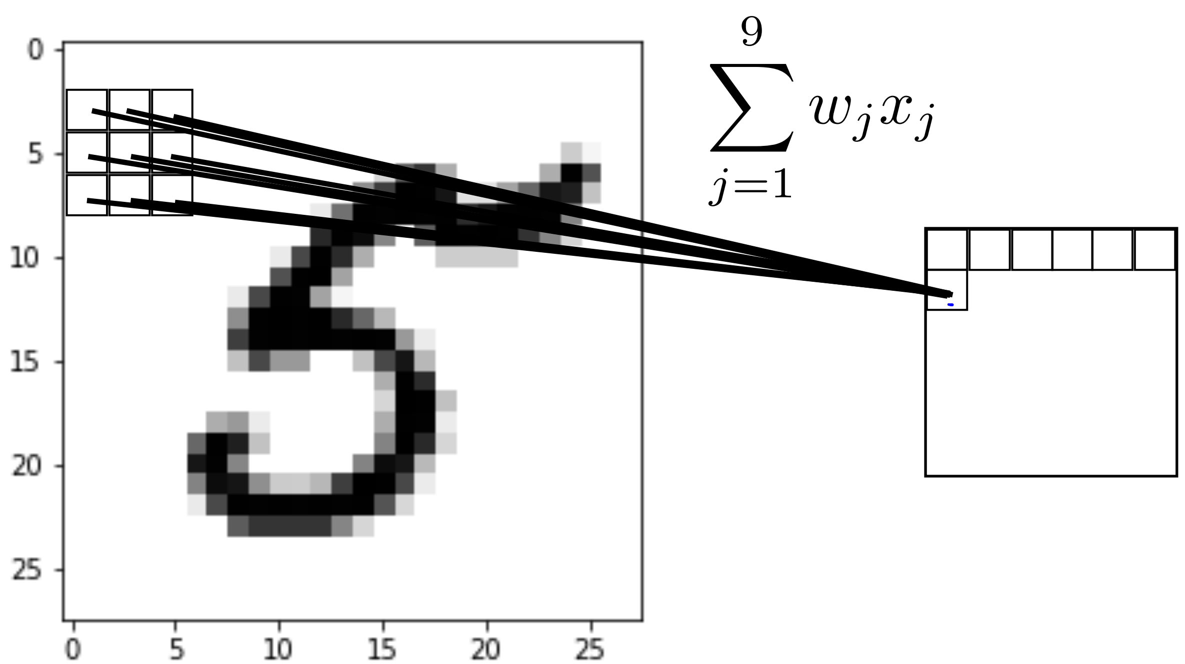
Weight Sharing

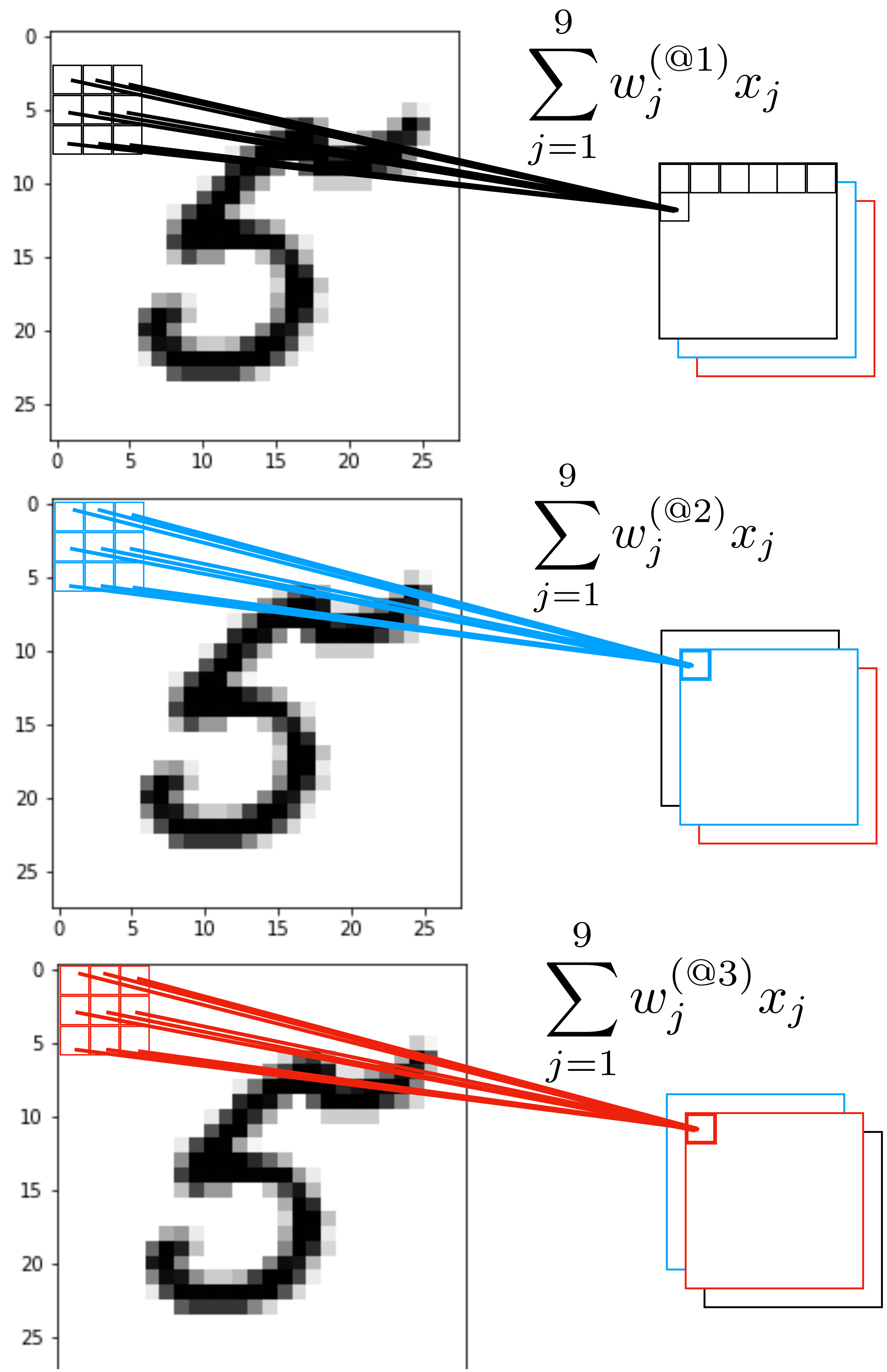
A "feature detector" (kernel) slides over the inputs to generate a feature map



Weight Sharing

A "feature detector" (kernel) slides over the inputs to generate a feature map





Multiple "feature detectors" (kernels) are used to create multiple feature maps

Size Before and After Convolutions

Feature map size:

input width

kernel width

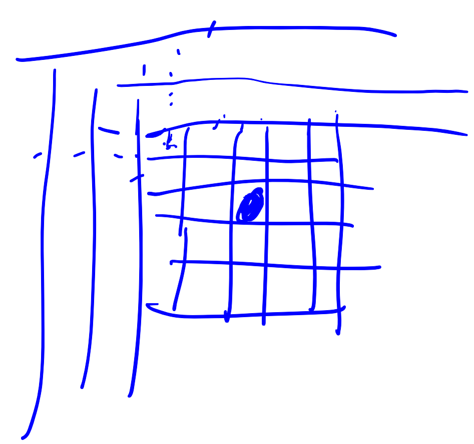
padding

$$O = \frac{W - K + 2P}{S} + 1$$

output width

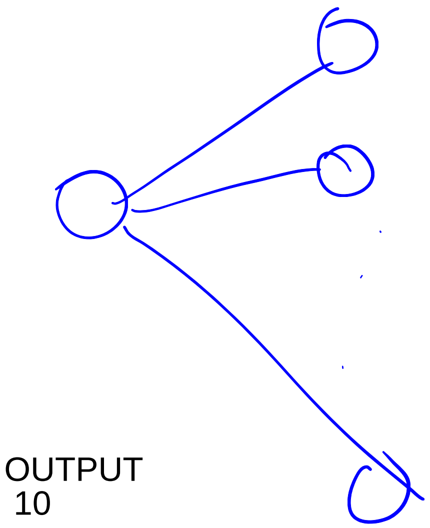
stride

$$\frac{32-5}{1} + 1 = 28$$



padding

Hidden Layers



PROC. OF THE IEEE, NOVEMBER 1998 7

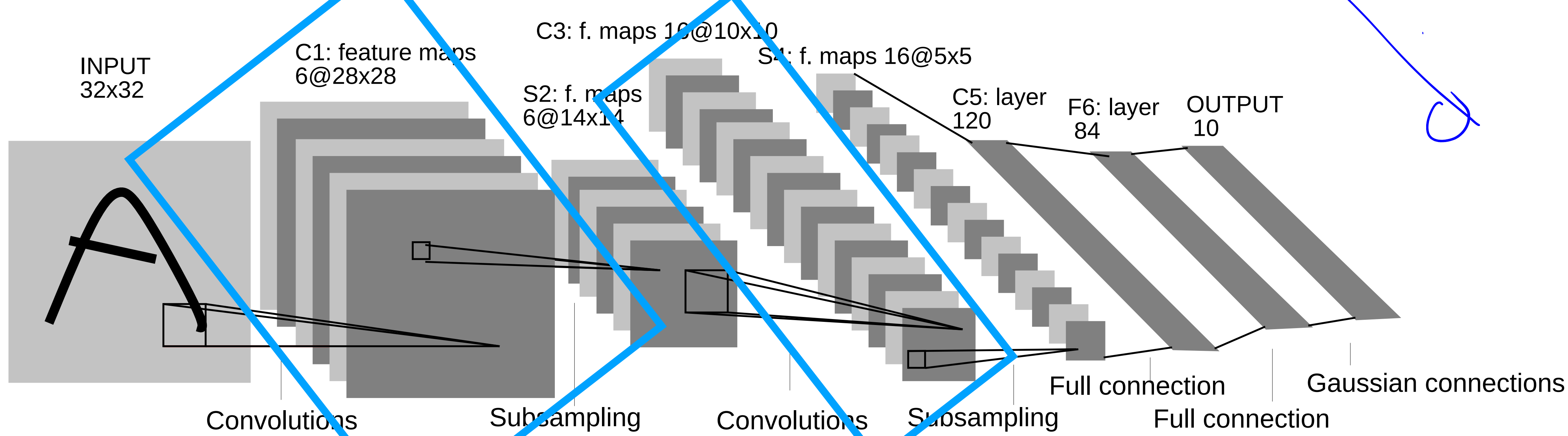


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

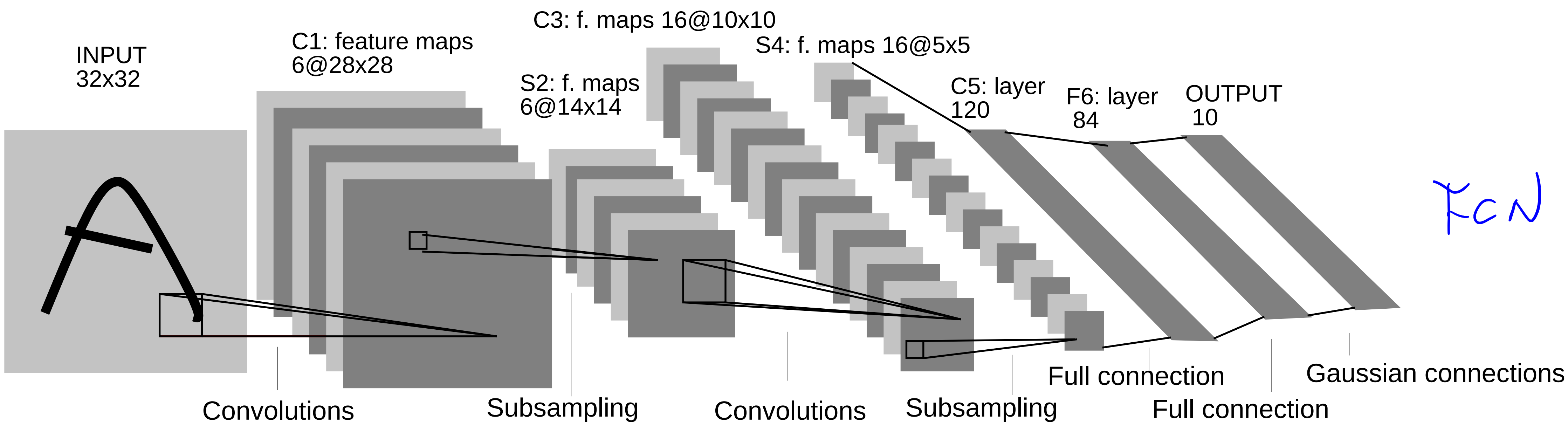
Each "bunch" of feature maps represents one hidden layer in the neural network.

Counting the FC layers, this network has 5 layers

Hidden Layers

nonlinear
(activation)

PROC. OF THE IEEE, NOVEMBER 1998 7

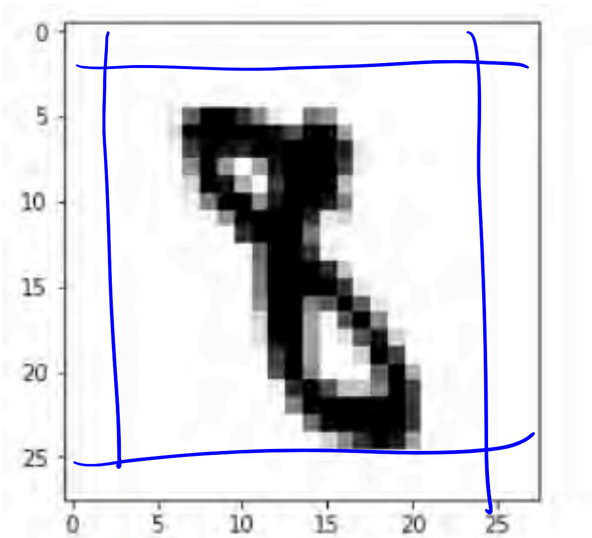


FCN

"Automatic feature extractor"

"Regular classifier"

Kernel Dimensions




```
a.shape
```

```
(1, 28, 28)
```

```
import torch
```

```
conv = torch.nn.Conv2d(in_channels=1,
                       out_channels=8,
                       kernel_size=(5, 5),
                       stride=(1, 1))
```

```
conv.weight.size()
```

```
torch.Size([8, 1, 5, 5])
```

```
conv.bias.size()
```

```
torch.Size([8])
```

For a grayscale image with a 5x5 feature detector (kernel), we have the following dimensions (number of parameters to learn)

What do you think is the output size for this 28x28 image?

$28 \times 28 \longrightarrow 24 \times 24$

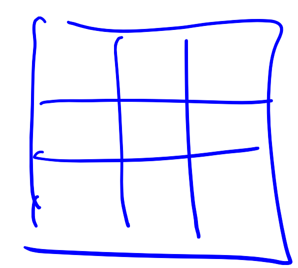
Backpropagation in CNNs

MLP

$\delta_3 \leftarrow \quad \delta_2 \leftarrow \quad \delta_1 \leftarrow$

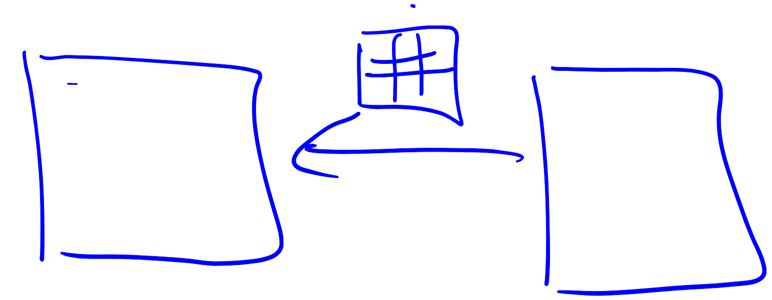
derivatives

Same overall concept as before: Multivariable chain rule, but now with an additional weight sharing constraint

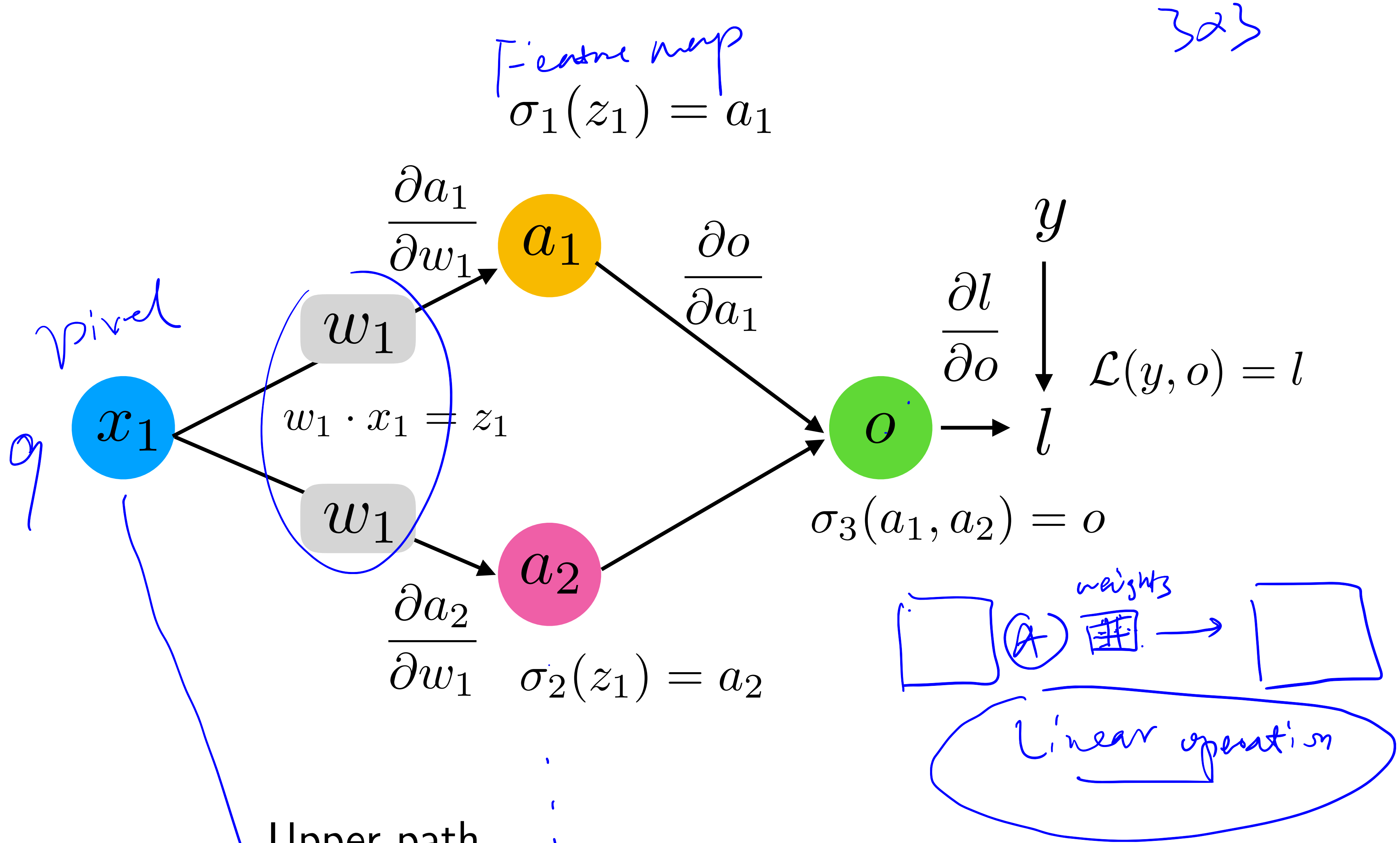


update kernel parameters

?



Remember Graph with Weight Sharing



Upper path

$$\frac{\partial l}{\partial w_1} = \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1} + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_1} \quad \text{(multivariable chain rule)}$$

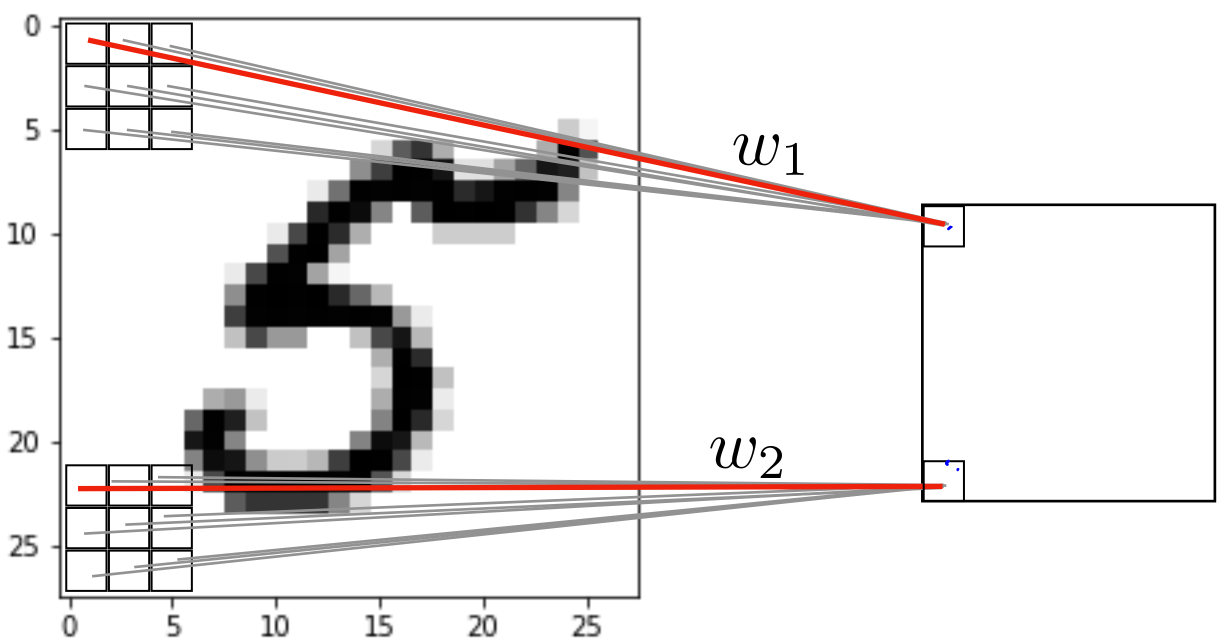
Lower path

w_2

Backpropagation in CNNs

Same overall concept as before: Multivariable chain rule, but now with an additional weight sharing constraint

Due to weight sharing: $w_1 = w_2$



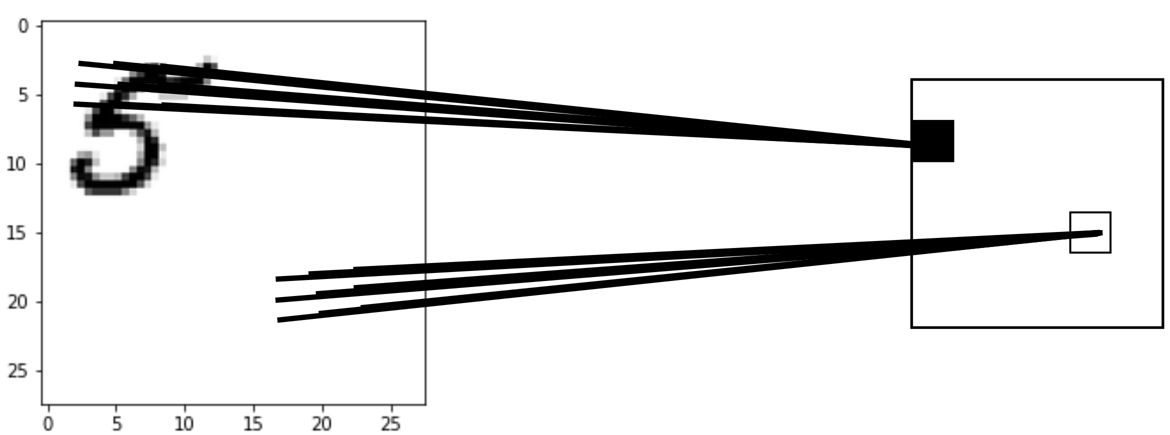
Optional averaging

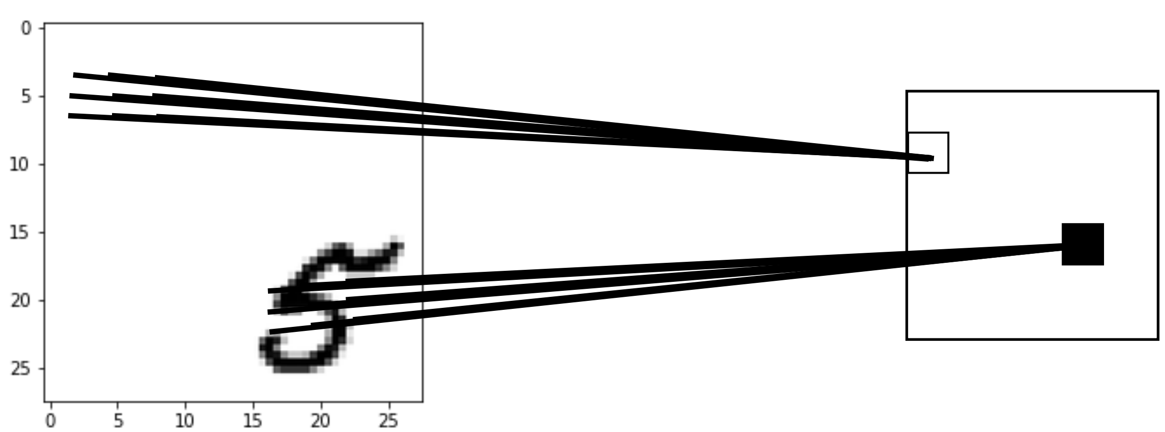
weight update:

$$w_1 := w_2 := w_1 - \eta \cdot \frac{1}{2}\left(\frac{\partial \mathcal{L}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial w_2}\right)$$

CNNs and Translation/Rotation/Scale Invariance

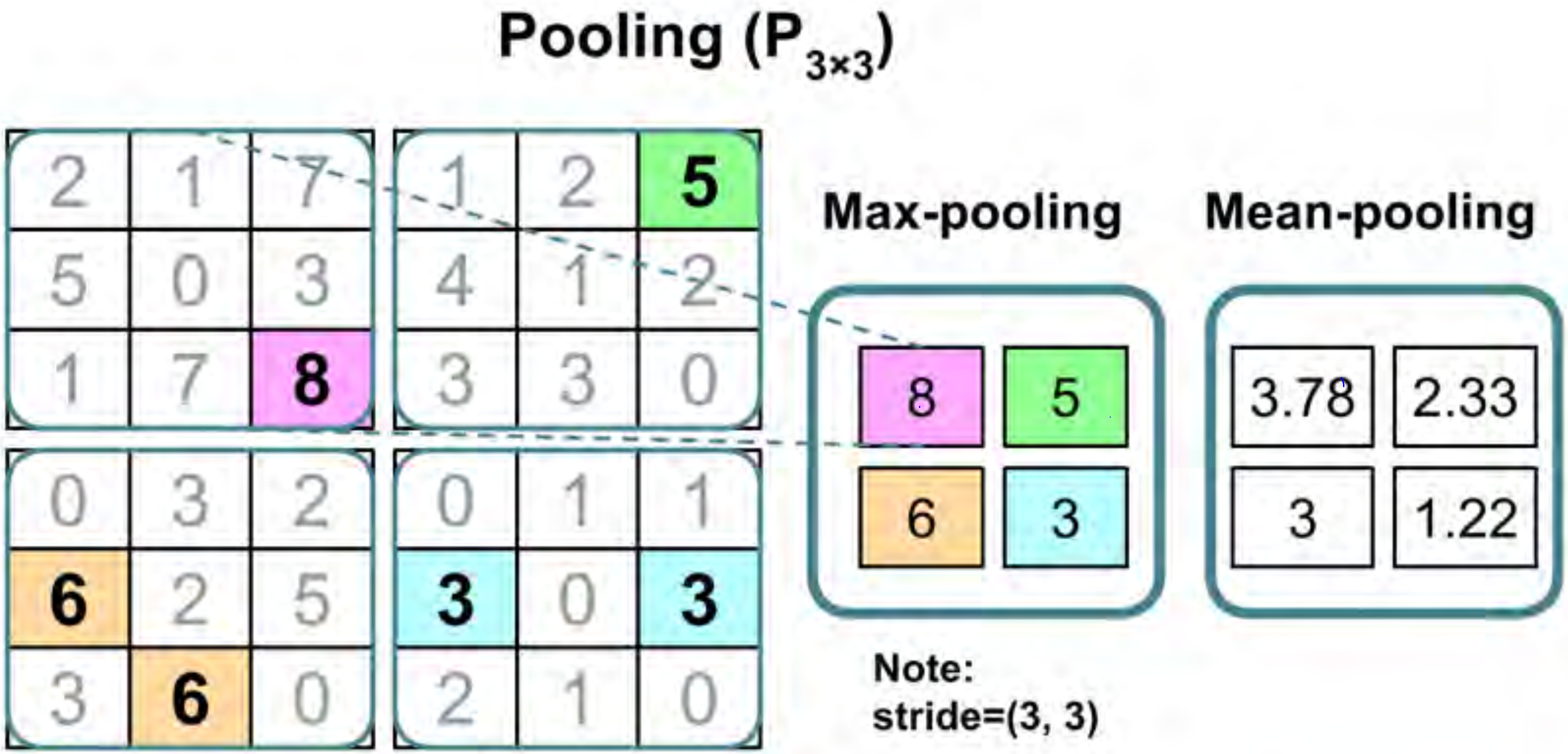
Note that CNNs are not really invariant to scale, rotation, translation, etc.





The activations are still dependent on the location, etc.

Pooling Layers Can Help With Local Invariance

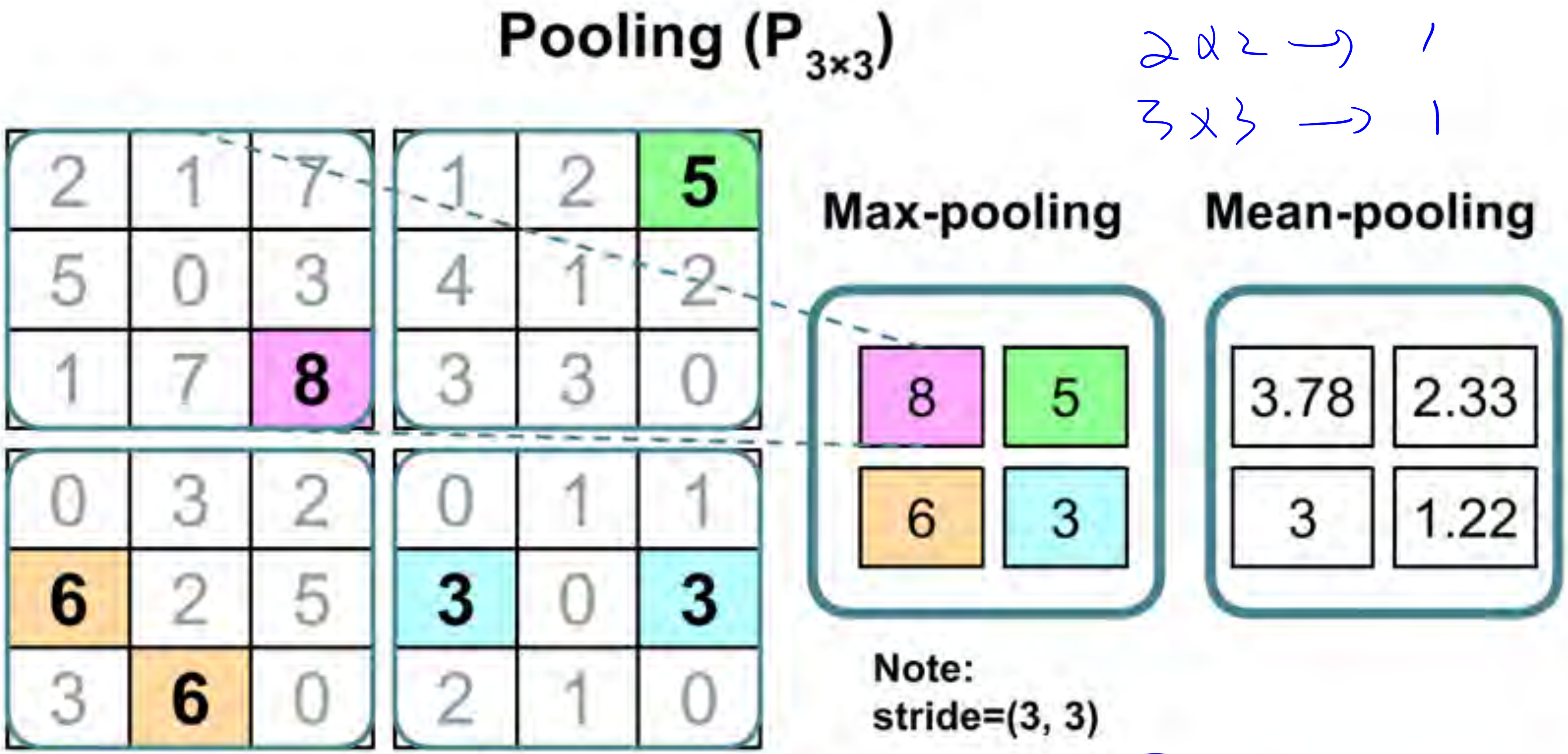


Downside: Information is lost.
May not matter for classification, but applications where relative position is important (like face recognition)

In practice for CNNs: some image preprocessing still recommended

Pooling Layers Can Help With Local Invariance

2x2 → 1
3x3 → 1



Note that typical pooling layers do not have any learnable parameters

Downside: Information is lost.

May not matter for classification, but applications where relative position is important (like face recognition)

In practice for CNNs: some image preprocessing still recommended

Main Breakthrough for CNNs:

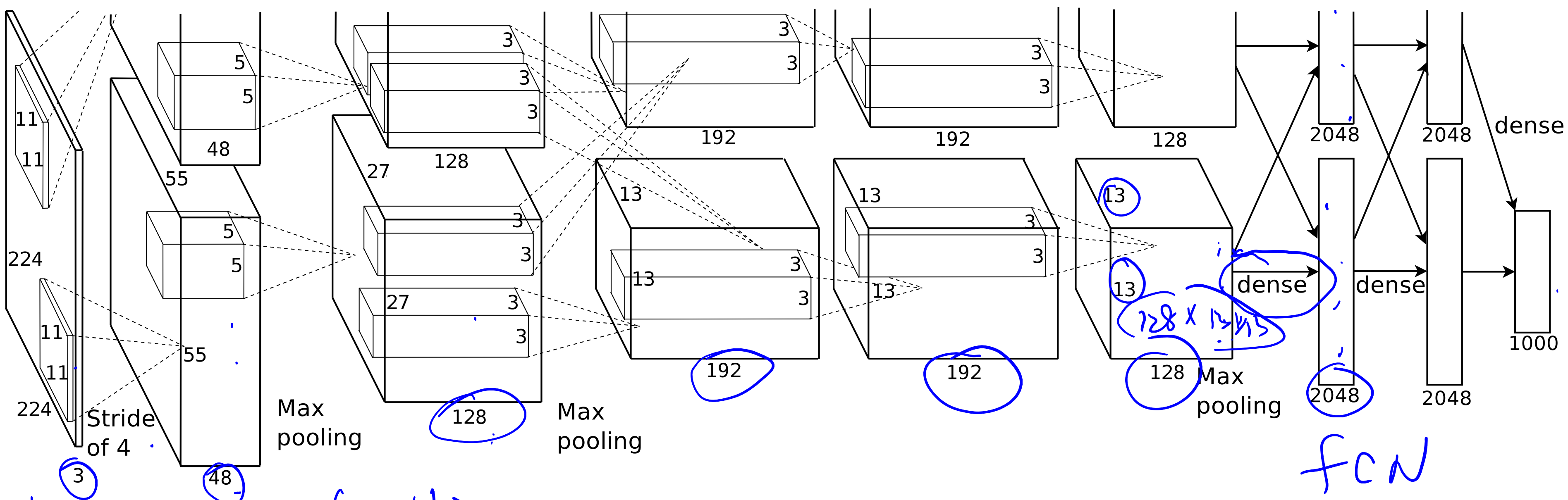
AlexNet & ImageNet

network dataset (1000 classes)

2012

GPU1 4096

ResNet



7x7

Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

nonlinear

ReLu

image pixel >,)

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097-1105).

Main Breakthrough for CNNs:
AlexNet & ImageNet

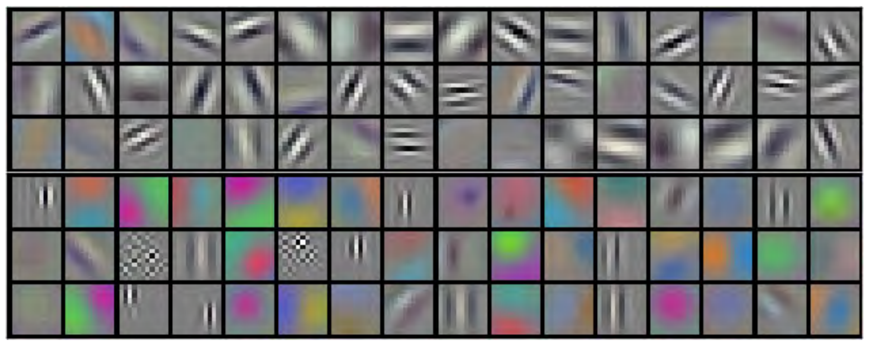


Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

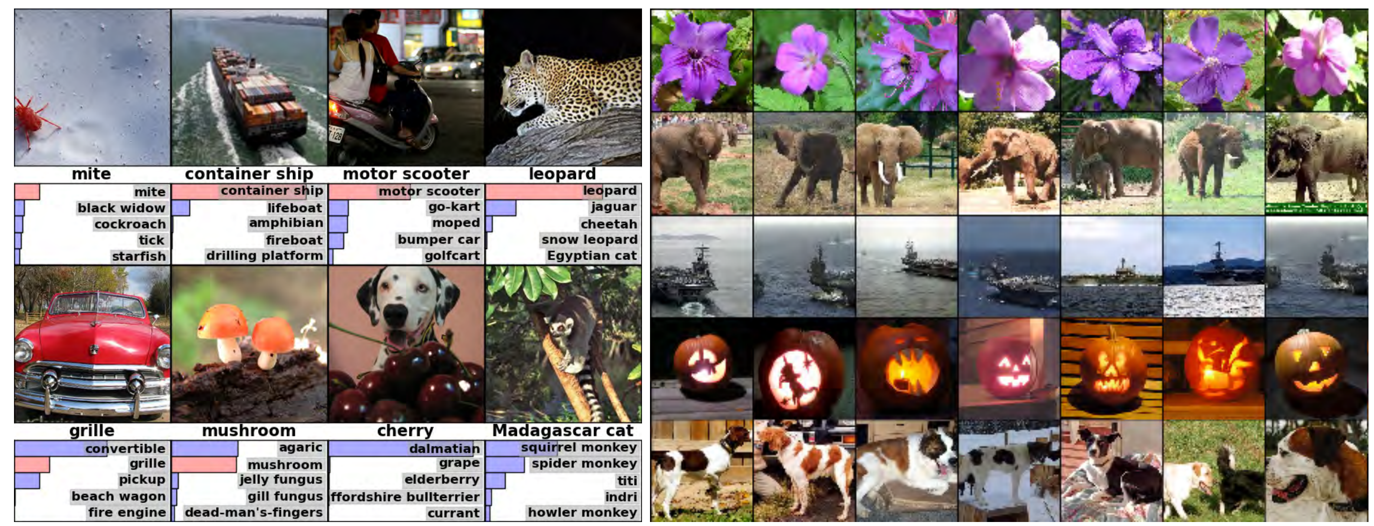
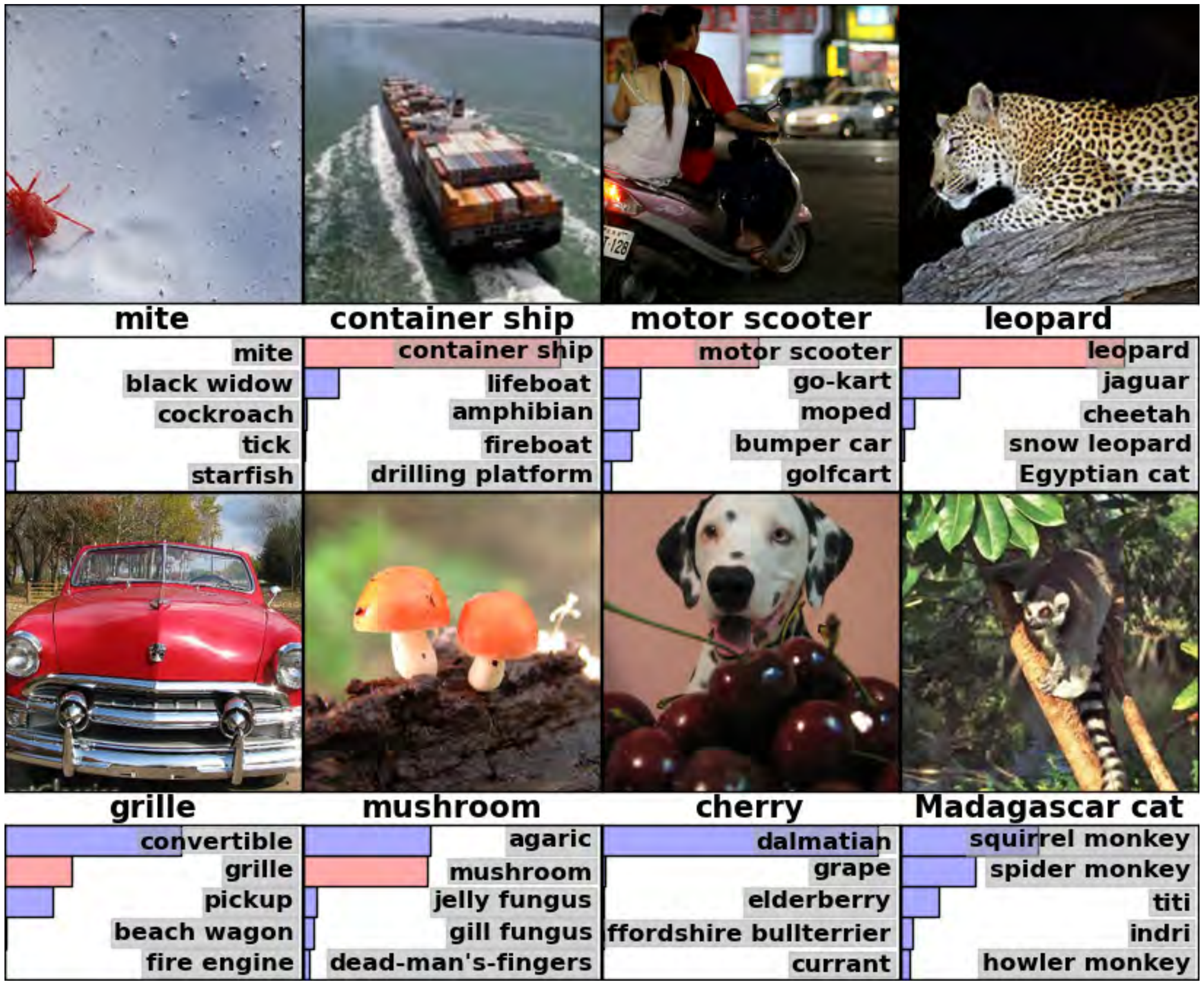


Figure 4: **(Left)** Eight ILSVRC-2010 test images and the five labels considered most probable by our model. The correct label is written under each image, and the probability assigned to the correct label is also shown with a red bar (if it happens to be in the top 5). **(Right)** Five ILSVRC-2010 test images in the first column. The remaining columns show the six training images that produce feature vectors in the last hidden layer with the smallest Euclidean distance from the feature vector for the test image.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097-1105).

Main Breakthrough for CNNs: AlexNet & ImageNet

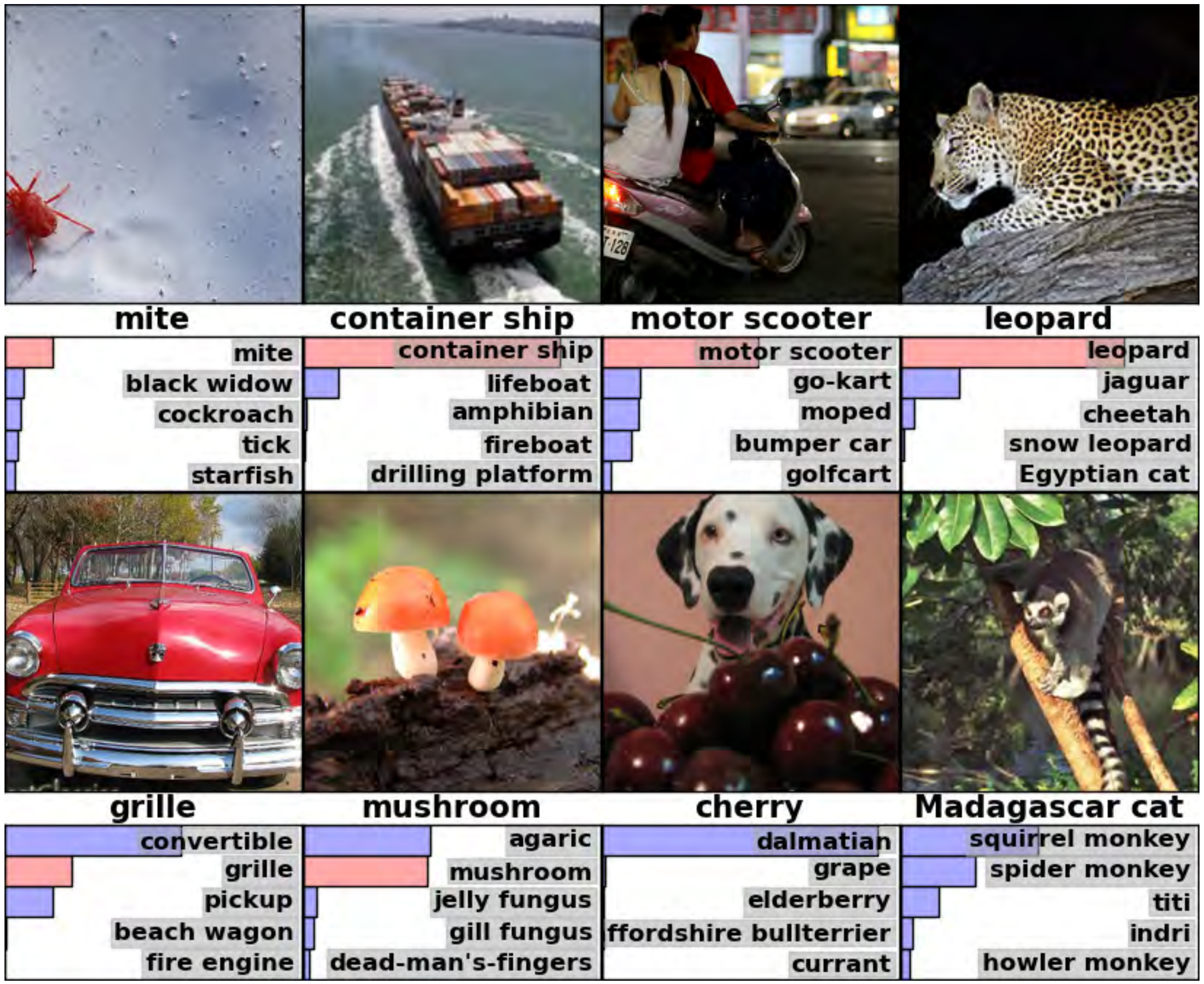


The ImageNet set that was used has ~1.2 million images and 1000 classes

Accuracy is measured as top-5 performance:
Correct prediction if the true label matches one of the top 5 predictions of the model

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097-1105).

Main Breakthrough for CNNs: AlexNet & ImageNet



Note that the actual network inputs were still 224x224 images (random crops from downsampled 256x256 images)

224x224 is still a good/ reasonable size today (224*224*3 = 150,528 features)

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097-1105).

Common CNN Architectures

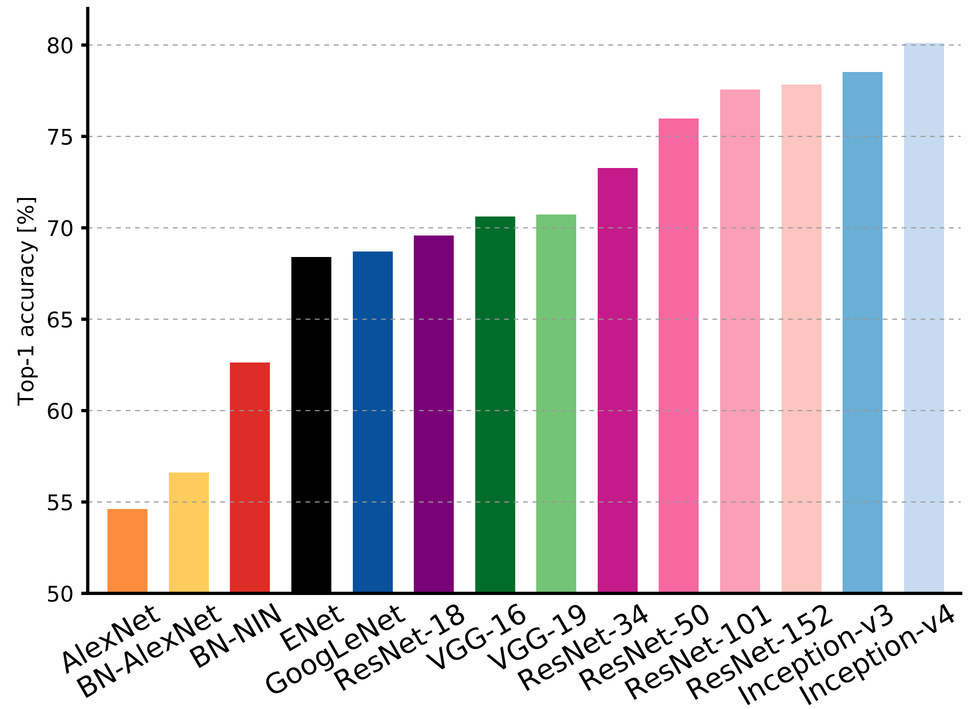


Figure 1: **Top1 *vs.* network.** Single-crop top-1 validation accuracies for top scoring single-model architectures. We introduce with this chart our choice of colour scheme, which will be used throughout this publication to distinguish effectively different architectures and their correspondent authors. Notice that networks of the same group share the same hue, for example ResNet are all variations of pink.

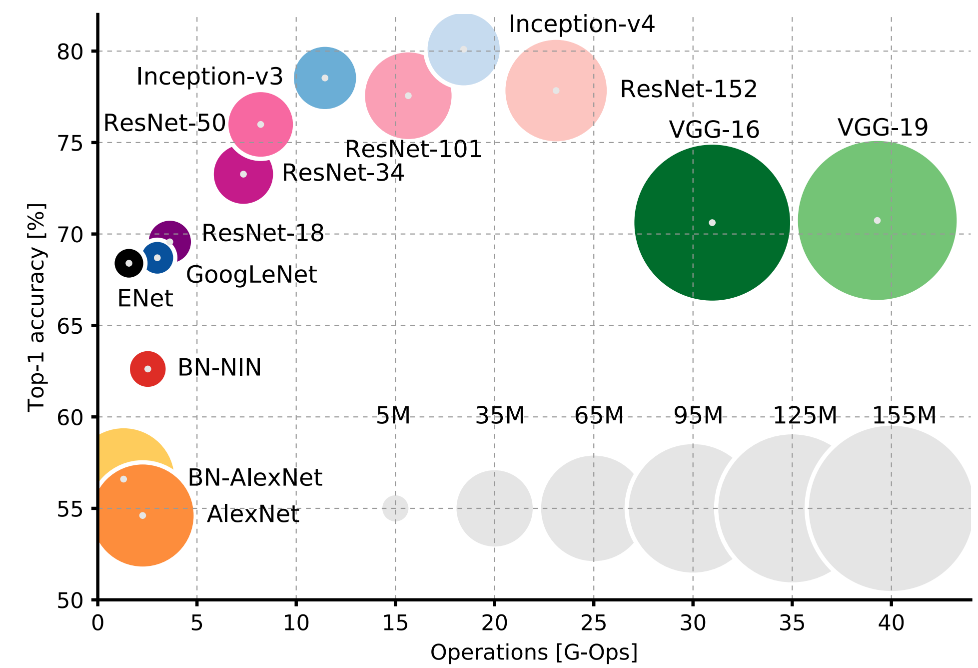


Figure 2: **Top1 *vs.* operations, size $\propto$ parameters.** Top-1 one-crop accuracy versus amount of operations required for a single forward pass. The size of the blobs is proportional to the number of network parameters; a legend is reported in the bottom right corner, spanning from 5×10^6 to 155×10^6 params. Both these figures share the same *y*-axis, and the grey dots highlight the centre of the blobs.

Canziani, A., Paszke, A., & Culurciello, E. (2016). An analysis of deep neural network models for practical applications. *arXiv preprint arXiv:1605.07678*.

Convolutions with Color Channels

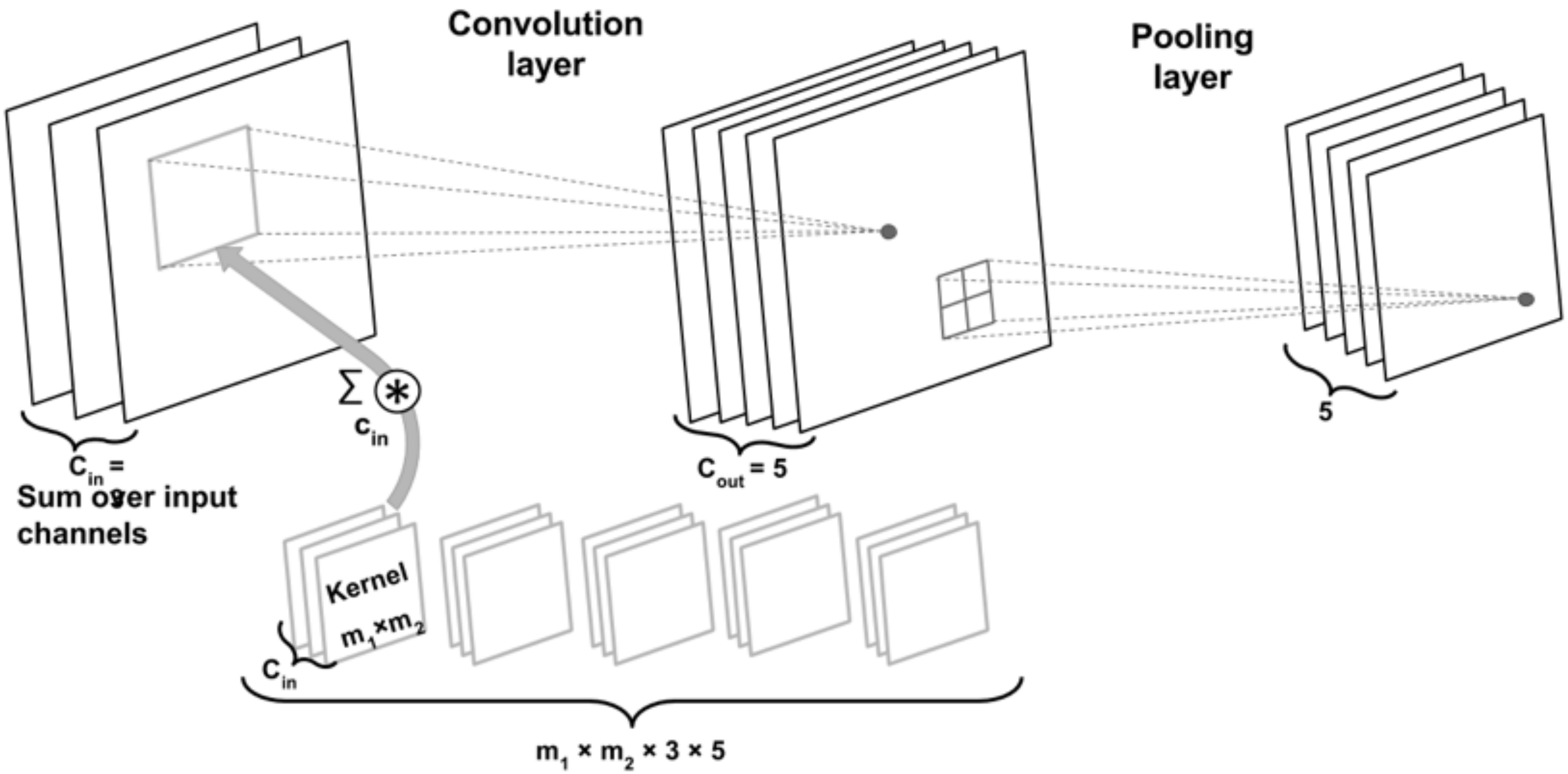


Image dimension: $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2 \times c_{in}}$ in NWHC format,
CUDA & PyTorch use NCWH

Convolutions with Color Channels

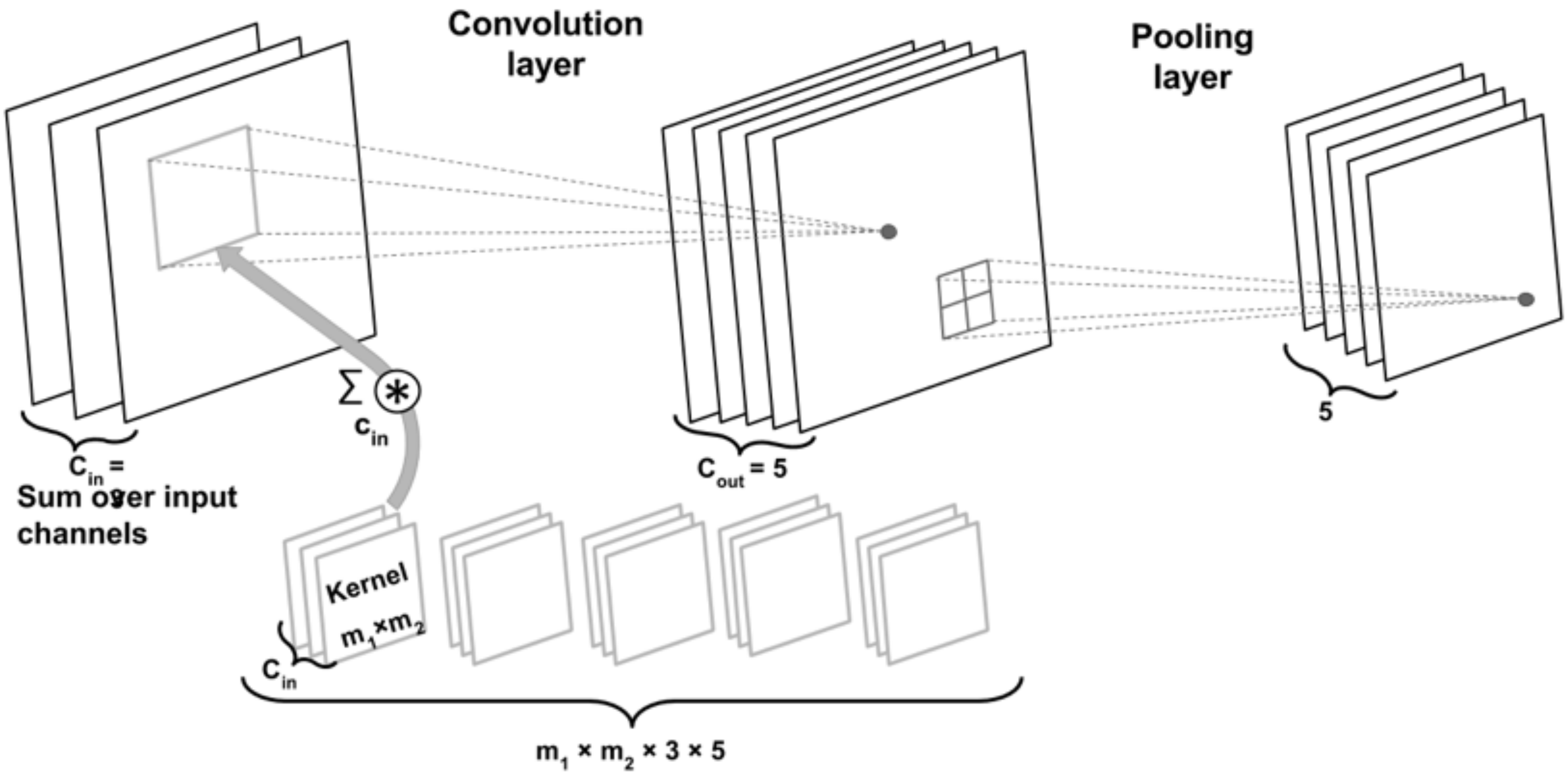
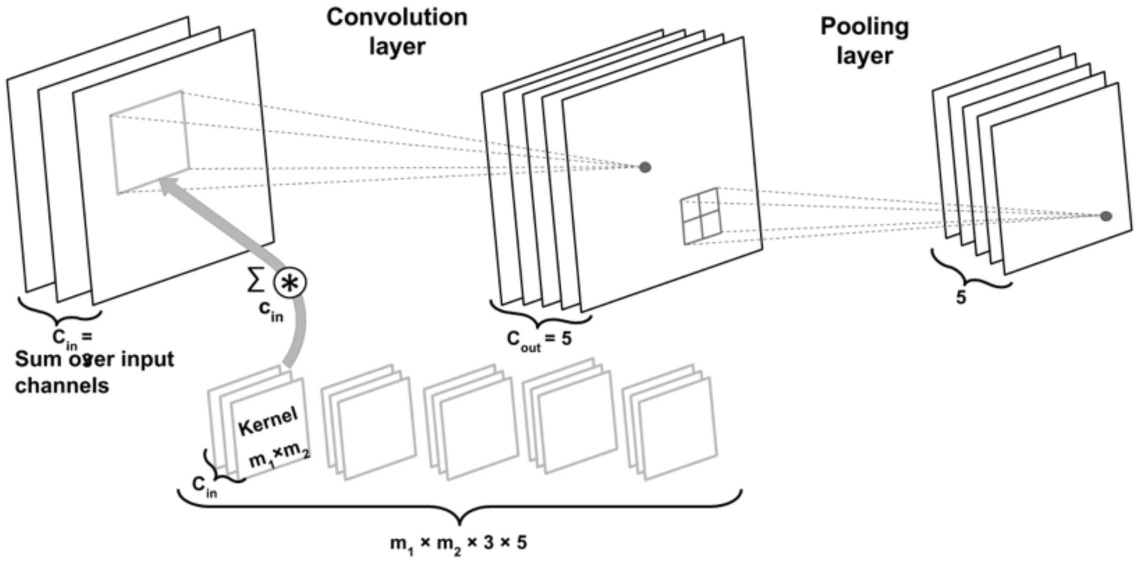


Image dimensions: $\mathbf{X} \in \mathbb{R}^{n_1 \times n_2 \times c_{in}}$

Kernel dimensions: $\mathbf{W} \in \mathbb{R}^{m_1 \times m_2 \times c_{in} \times c_{out}}$ $\quad \mathbf{b} \in \mathbb{R}^{c_{out}}$

Convolutions with Color Channels



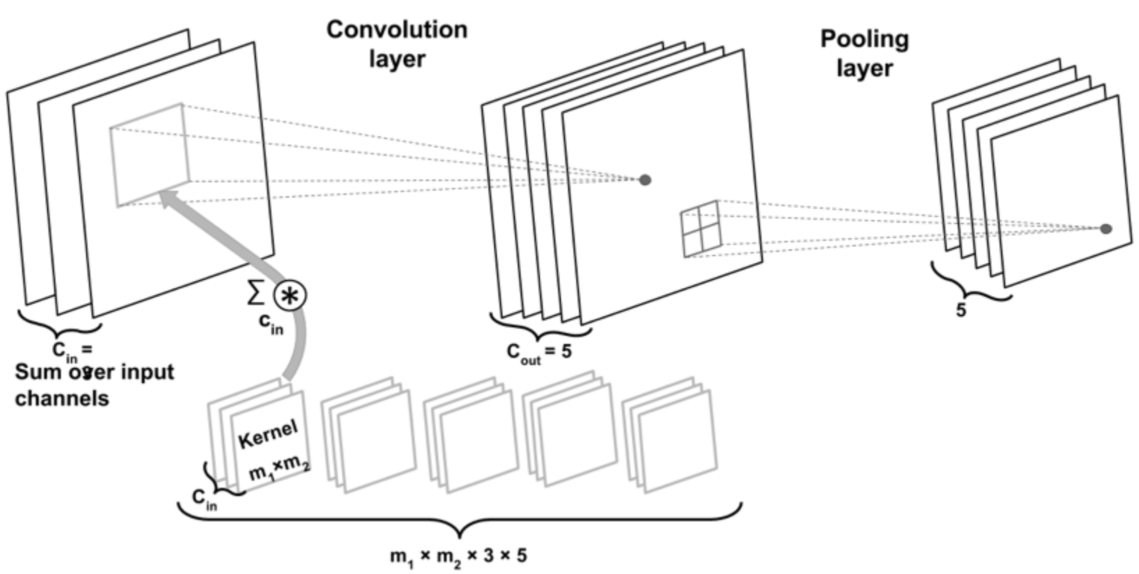
Number of parameters:

$$m_1 \times m_2 \times 3 \times 5 + 5$$

Assume 5x5 kernel:

$$5 \times 5 \times 3 \times 5 + 5 = 380$$

Convolutions with Color Channels



Assume "same" padding (more on padding later), such that the hidden layer has the same height and width as the original images

<u>If we use a CNN:</u>

Number of parameters:

$$m_1 \times m_2 \times 3 \times 5 + 5$$

Assume 128x128 images and 5x5 kernel:

$$5 \times 5 \times 3 \times 5 + 5 = 380$$

<u>If we use a fully connected layer:</u>

Number of parameters:

$$(n_1 \times n_2 \times 3) \times (n_1 \times n_2 \times 5) + (n_1 \times n_2 \times 5)$$

$$(n_1 \times n_2)^2 \times 3 \times 5 + n_1 \times n_2 \times 5$$

$$(128 \times 128)^2 \times 3 \times 5 + 128 \times 128 \times 5$$

$$= 4,026,613,760$$

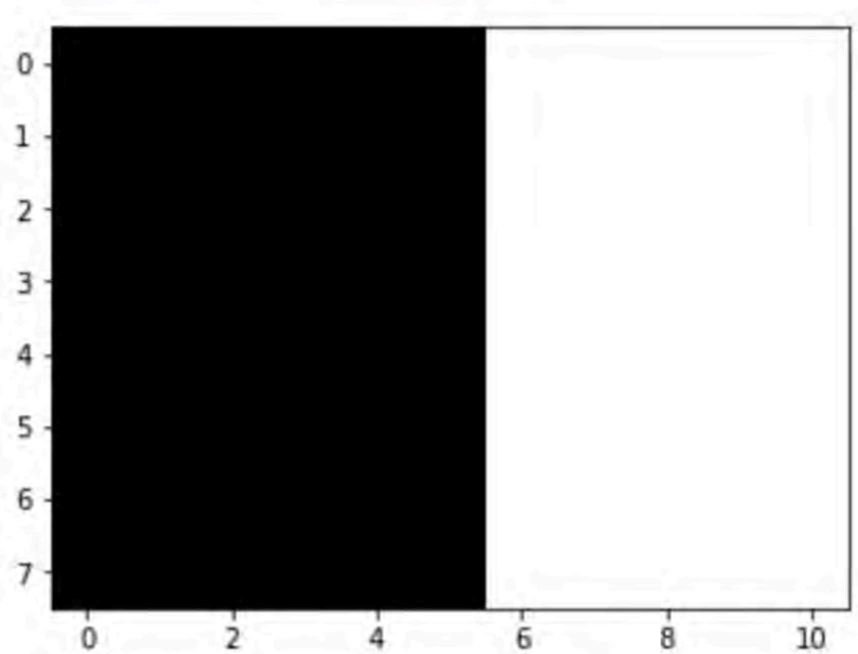
What a CNN Can See

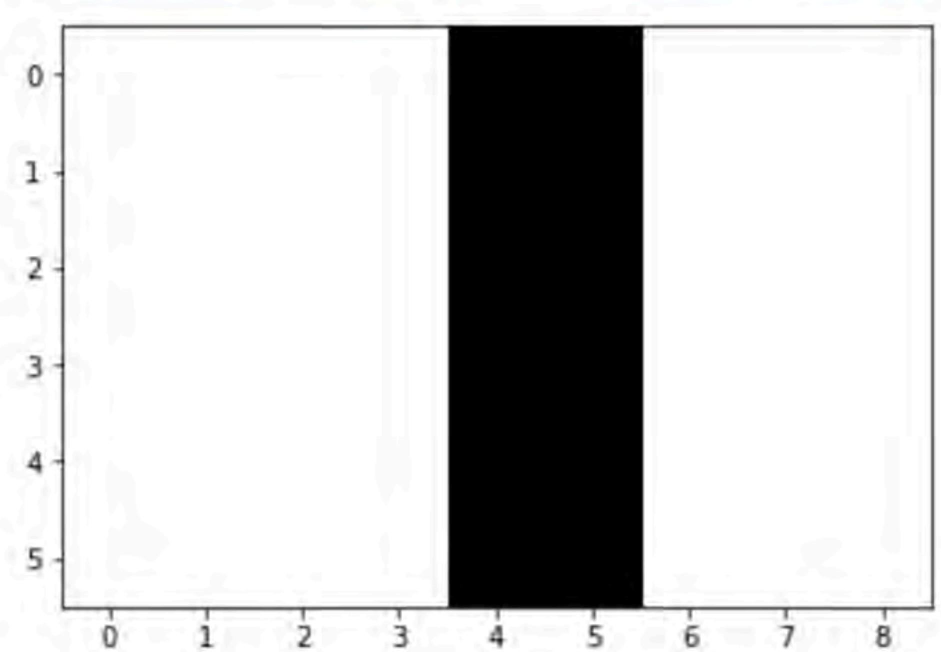
Simple example: vertical edge detector

```
conv.weight[0, 0, :, :] = torch.tensor([[1, 0, -1],
                                         [1, 0, -1],
                                         [1, 0, -1]]).float()

t = torch.tensor([
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
[0., 0., 0., 0., 0., 0., 1., 1., 1., 1., 1.],
])

plt.imshow(t, cmap='gray');
```

(From classical computer vision research)

```
tt = torch.zeros([1, 1] + list(t.size()))
tt[0, 0, :, :] = t
after = conv(tt)
plt.imshow(after[0, 0, :, :].detach().numpy(), cmap='gray');
```





What a CNN Can See

Simple example: vertical edge detector

```
conv = torch.nn.Conv2d(in_channels=1,
                       out_channels=1,
                       kernel_size=(3, 3))
```

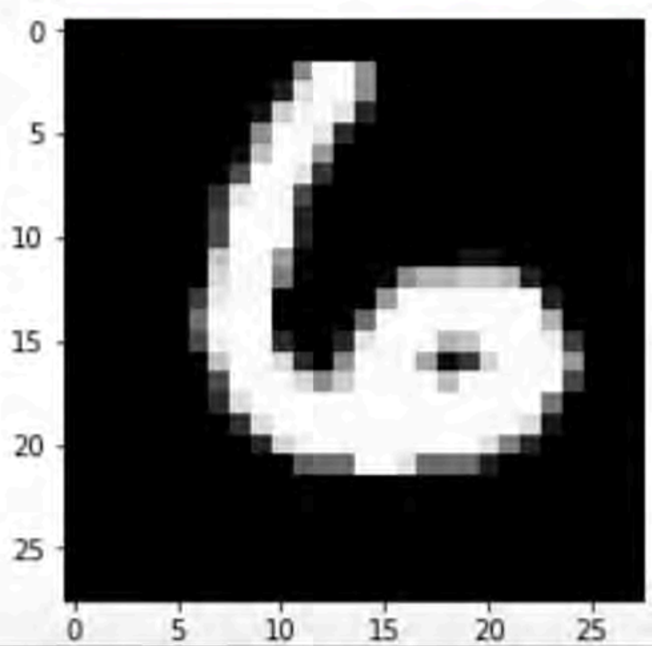
```
conv.weight.size()
```

```
torch.Size([1, 1, 3, 3])
```

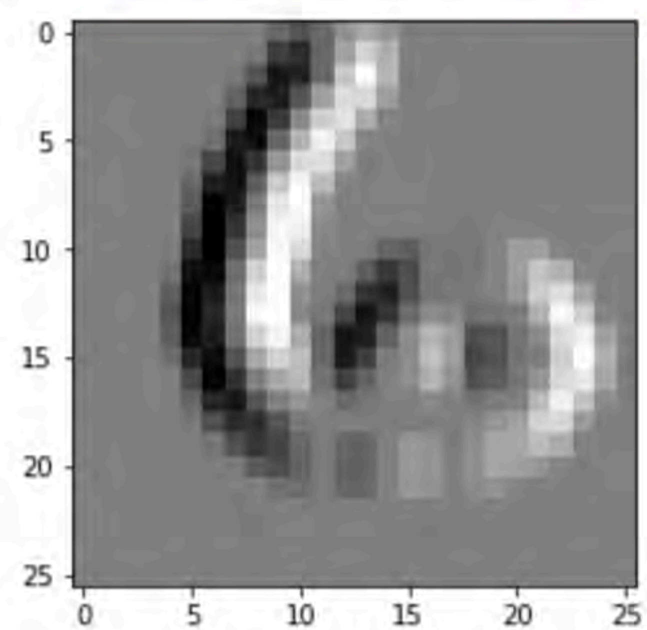
```
conv.weight[0, 0, :, :] = torch.tensor([[1, 0, -1],
                                        [1, 0, -1],
                                        [1, 0, -1]]).float()
conv.bias[0] = torch.tensor([0.]).float()
```

```
images_after = conv(images)
```

```
plt.imshow(images[5, 0], cmap='gray');
```




```
plt.imshow(images_after[5, 0].detach().numpy() , cmap='gray');
```



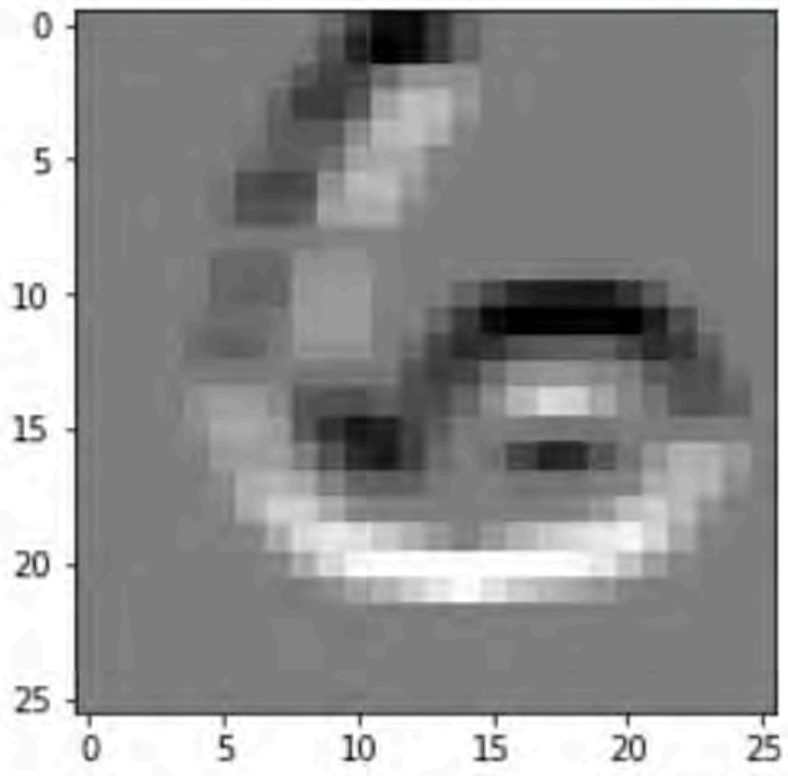
What a CNN Can See

Simple example: horizontal edge detector

```
conv.weight[0, 0, :, :] = torch.tensor([[1, 1, 1],
                                        [0, 0, 0],
                                        [-1, -1, -1]]).float()
conv.bias[0] = torch.tensor([0.]).float()
```

```
images_after2 = conv(images)
```

```
plt.imshow(images_after2[5, 0].detach().numpy() , cmap='gray');
```



A CNN can learn whatever it finds best based on optimizing the objective (e.g., minimizing a particular loss to achieve good classification accuracy)

What a CNN Can See

Which patterns from the training set activate the feature map?

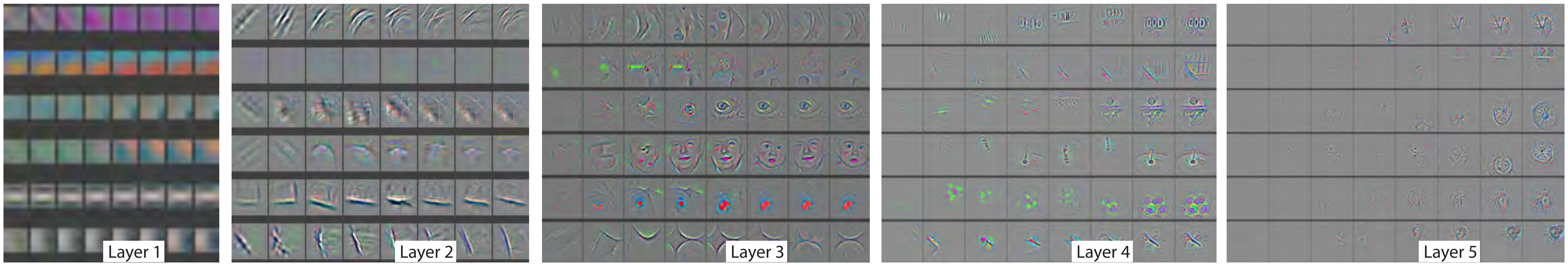


Fig. 4. Evolution of a randomly chosen subset of model features through training. Each layer's features are displayed in a different block. Within each block, we show a randomly chosen subset of features at epochs [1,2,5,10,20,30,40,64]. The visualization shows the strongest activation (across all training examples) for a given feature map, projected down to pixel space using our deconvnet approach. Color contrast is artificially enhanced and the figure is best viewed in electronic form.

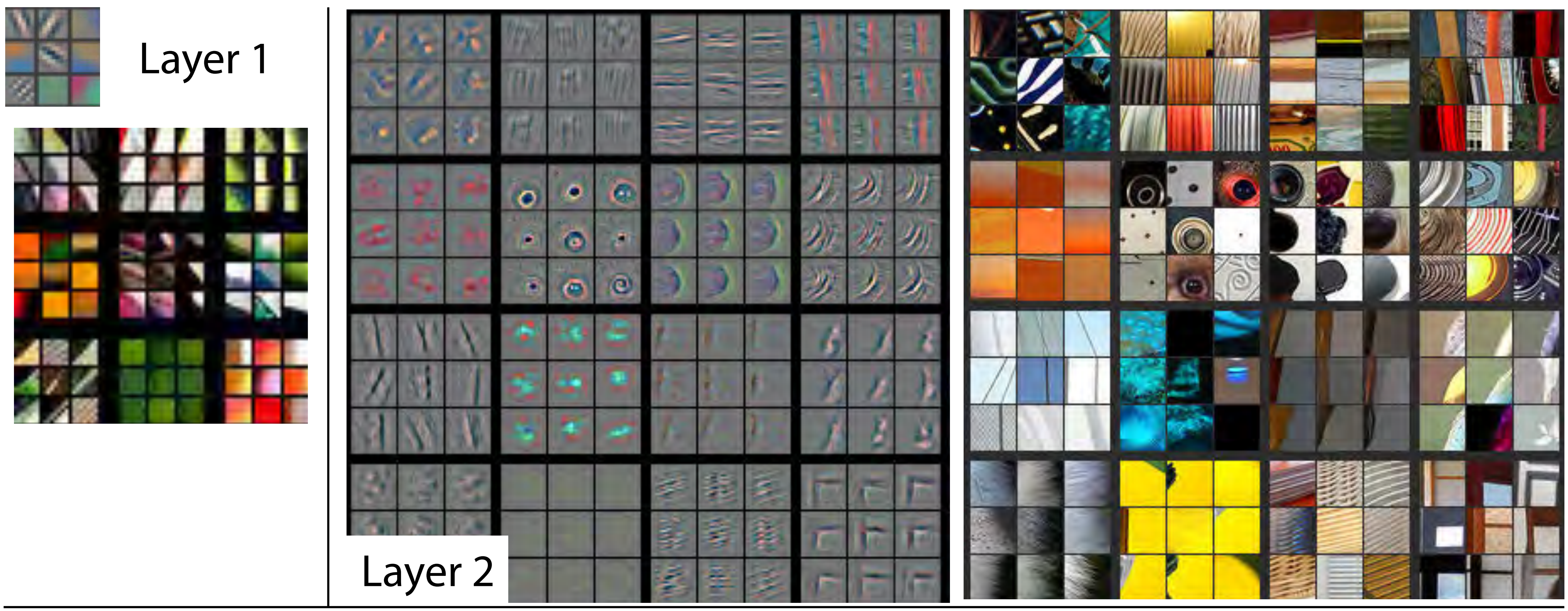
Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In *European conference on computer vision* (pp. 818-833). Springer, Cham.

Method: backpropagate strong activation signals in hidden layers to the input images, then apply "unpooling" to map the values to the original pixel space for visualization

What a CNN Can See

Which patterns from the training set activate the feature map?

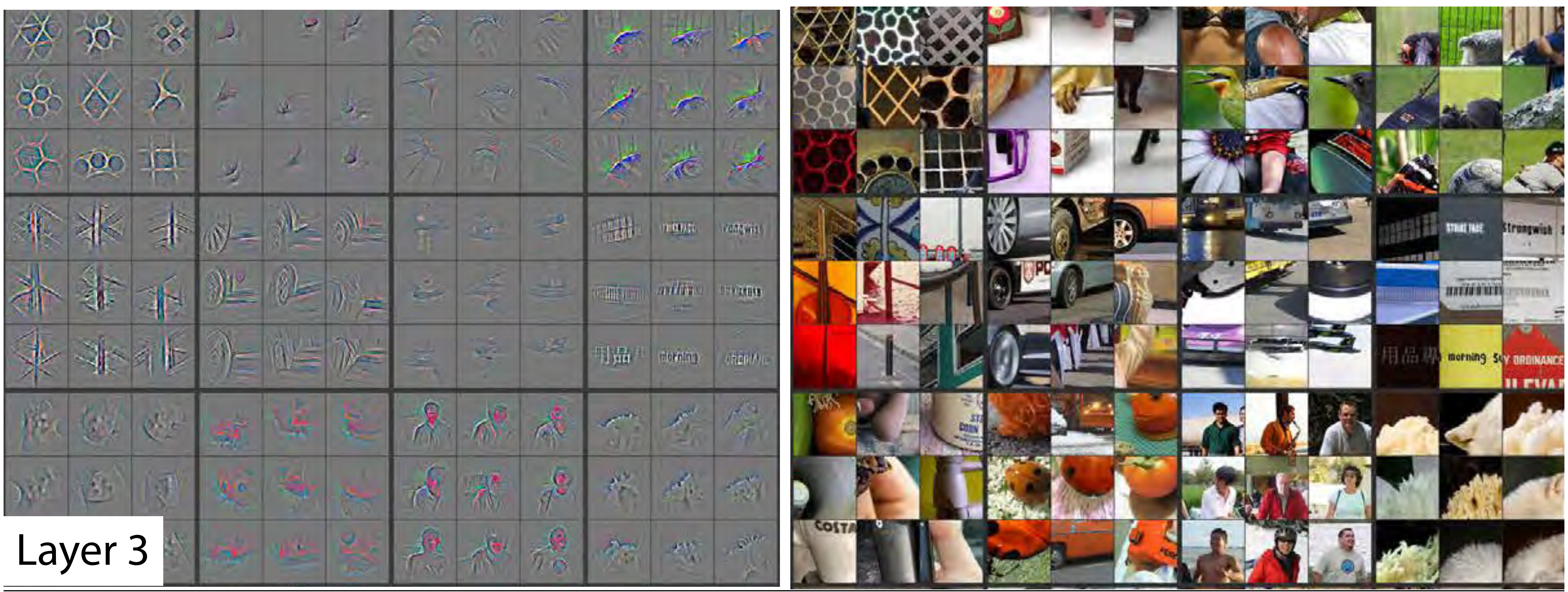
Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In *European conference on computer vision* (pp. 818-833). Springer, Cham.



What a CNN Can See

Which patterns from the training set activate the feature map?

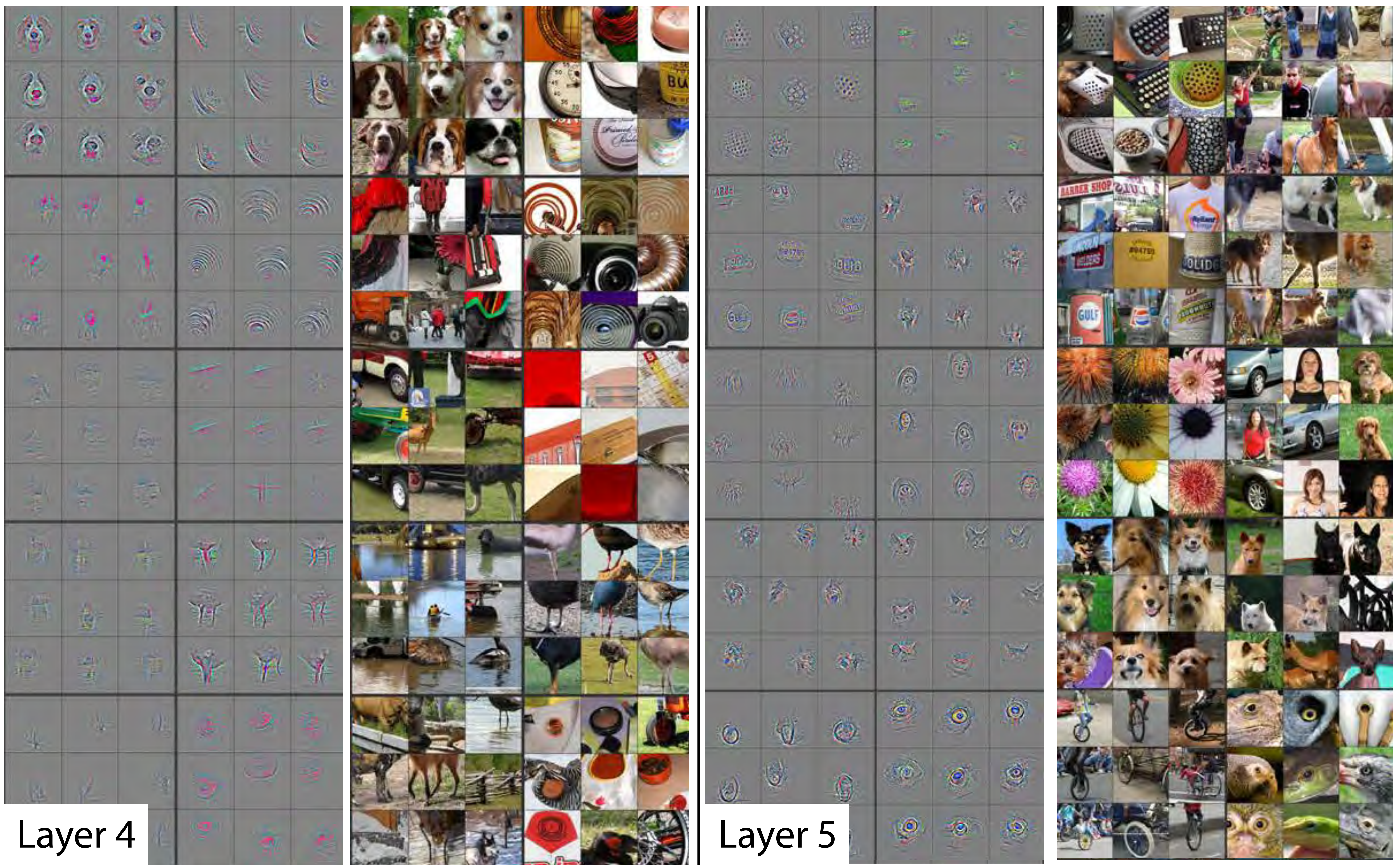
Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In *European conference on computer vision* (pp. 818-833). Springer, Cham.



What a CNN Can See

Which patterns from the training set activate the feature map?

Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In *European conference on computer vision* (pp. 818-833). Springer, Cham.



End of part 1